Mini project report on

LEGAL EVIDENCE CHAIN-OF-CUSTODY AND INTELLIGENT CASE INVESTIGATION SYSTEM

(ChainSecure)

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE23CS351A – DBMS Project

Submitted by:

Sindhu GV	**PES2UG24CS661**
Vanishree	**PES2UG24CS665**

under the guidance of

Dr. Shilpa S
Assistant Professor

PES University
AUG – DEC 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PES UNIVERSITY
(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

Legal Evidence Chain-of-Custody and Intelligent Case Investigation System

is a bonafide work carried out by

Sindhu GV	**PES2UG24CS661**
Vanishree	**PES2UG24CS665**

in partial fulfilment for the completion of fifth semester DBMS Project (UE23CS351A) in the Program of Study – Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period AUG. 2026 – DEC. 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 5th semester academic requirements in respect of project work.

Signature
Dr. Shilpa S
Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled **Legal Evidence Chain-of-Custody and Intelligent Case Investigation System** has been carried out by us under the guidance of **Dr. Shilpa S, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester AUG – DEC 2026.

Sindhu GV	**PES2UG24CS661**	Sindhu
Vanishree	**PES2UG24CS665**	Vanishree

ACKNOWLEDGEMENT

We would like to express our gratitude to Dr. Shilpa S, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE23CS351A – DBMS Project.

We take this opportunity to thank Dr. Sandesh B J, Professor and Chairperson, Department of Computer Science and Engineering, PES University, for all the knowledge and support we have received from the department.

We are deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, and Dr. Suryaprasad J, Vice-Chancellor, PES University, for providing us various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement we have received from our family and friends.

ABSTRACT

Digital and physical evidence is only useful in court if every handover from the moment of collection can be proved. A single undocumented transfer or an unverifiable hash can make an otherwise strong piece of evidence inadmissible. In many police and forensic units this history is still kept in paper registers or scattered spreadsheets, which are easy to lose, hard to audit, and impossible to query.

ChainSecure is a legal evidence chain-of-custody and case investigation system that records cases, evidence, forensic tests, people and every custody transfer in a relational database. MySQL 8 is the system of record and enforces integrity through primary and foreign keys, unique, check and enumeration constraints, indexes, a trigger that automatically marks evidence as in transit after each handover, a stored function that verifies the continuity of the custody chain, a reporting view and transactional transfer logging. A second database, MongoDB, stores an append-only audit trail of important actions such as logins, logouts, evidence creation, deletion and chain verification.

The web application is built with Next.js and React. It provides secure login with bcrypt-hashed passwords and HTTP-only JWT session cookies, role-based access control for Admin, Officer and Forensic Officer users, a dashboard with live metrics, searchable record views, validated data-entry forms, chain verification and logout. This report documents the problem definition, ER model, relational mapping, DDL, DML, queries, advanced SQL objects and the front-end implementation of the system.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	**INTRODUCTION**	**10**
2.	**PROBLEM DEFINITION**	**12**
3.	**ER MODEL**	**14**
4.	**ER TO RELATIONAL MAPPING**	**17**
5.	**DDL STATEMENTS**	**20**
6.	**DML STATEMENTS**	**26**
7.	**QUERIES (SIMPLE QUERY AND UPDATE AND DELETE OPERATION, CORRELATED QUERY AND NESTED QUERY)**	**34**
8.	**STORED PROCEDURES, FUNCTIONS AND TRIGGERS**	**43**
9.	**FRONT END DEVELOPMENT**	**49**
	REFERENCES / BIBLIOGRAPHY	**57**
	APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS	**58**

LIST OF TABLES

Table No.	Title	Page No.
1.1	Technology stack	11
1.2	Team member responsibilities	11
2.1	User roles and permissions	13
3.1	Entities and their attributes (PK = primary key, FK = foreign key)	14
3.2	Relationships between entities	14
4.1	Application of the ER-to-relational mapping algorithm	17
4.2	Relations, primary keys and foreign keys	19
5.1	Constraints used in the schema	25
9.1	REST endpoints of the application	49
9.2	Application test cases	55

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	Entity–Relationship diagram of ChainSecure	15
4.1	Relational schema of ChainSecure	18
5.1	Database creation and SHOW TABLES output	20
5.2	DDL of evidence_category table	20
5.3	DDL of location table	21
5.4	DDL of user table	21
5.5	DDL of case table	22
5.6	DDL of person table	22
5.7	DDL of involves table	23
5.8	DDL of evidence table	24
5.9	DDL of belongs_to table	24
5.10	DDL of forensic_test table	24
5.11	DDL of custody_transfer table	25
6.1	Records inserted into evidence_category	26
6.2	Records inserted into location	27
6.3	Records inserted into user	27
6.4	Records inserted into case	28
6.5	Records inserted into person	29
6.6	Records inserted into involves	29
6.7	Records inserted into evidence	30
6.8	Records inserted into belongs_to	31
6.9	Records inserted into forensic_test	32
6.10	Records inserted into custody_transfer	33
7.1	(a) SELECT with WHERE clause	34
7.2	(b) ORDER BY	34
7.3	(c) JOIN of evidence, case and category	35
7.4	(d) JOIN of custody transfers with both users	36
7.5	(e) GROUP BY with COUNT	36
7.6	(f) GROUP BY with HAVING	37
7.7	(g) Evidence count per case (LEFT JOIN)	37
7.8	(h) COUNT, MIN and MAX together	38
7.9	(a) Update evidence status	38
7.10	(b) Update case status	39
7.11	Delete a forensic test record	39

Figure No.	Title	Page No.
7.12	(a) Latest evidence collected in each case	40
7.13	(b) Number of custody transfers for each evidence item	40
7.14	(a) Evidence belonging to open cases	41
7.15	(b) Users who have collected evidence	41
7.16	(c) Evidence collected after the earliest collection date	42
8.1	Creation of stored function verify_chain_integrity	44
8.2	Testing the function	44
8.3	Creation of trigger trg_after_custody_transfer	45
8.4	Testing the trigger	45
8.5	Transaction example	46
8.6	Creation and use of view evidence_summary	47
8.7	Indexes on evidence, custody_transfer and case	48
9.1	System architecture of ChainSecure	49
9.2	Login page	50
9.3	Login with invalid credentials	51
9.4	Overview dashboard (Admin)	51
9.5	Cases page	51
9.6	Evidence vault	52
9.7	Register evidence form	52
9.8	Delete confirmation and chain verification	52
9.9	Custody chain page	53
9.10	Log custody transfer form	53
9.11	Forensic tests page and add-test form	53
9.12	Audit log (Admin)	54
9.13	Users and roles (Admin)	54
9.14	Restricted view for Forensic Officer	54
9.15	Logout	55
9.16	MongoDB audit_events collection	55

1. INTRODUCTION

1.1 Overview

The chain of custody is the documented, chronological record of who held a piece of evidence, when, where, and why. Courts expect this record to be unbroken: if any link is missing or inconsistent, the defence can argue that the evidence may have been tampered with or replaced. As investigations increasingly involve digital material such as phones, CCTV footage and bank records alongside physical items, keeping this record accurate by hand becomes difficult.

ChainSecure is a database-driven web application that records each case, every item of evidence collected for it, the forensic tests carried out on that evidence, the people involved, and each custody transfer between officers and locations. The system uses MySQL as the primary relational store and MongoDB for an append-only audit log, exposed through a Next.js web interface with authentication and role-based authorisation.

1.2 Motivation

- Manual registers and spreadsheets cannot guarantee that the custody history is complete, ordered and tamper-evident.
- Investigators need quick answers to questions such as “Where is this item now?”, “Who handled it last?” and “Which tests were run on it?”.
- Forensic and legal reviews need proof of who changed what, and when, which requires an audit trail that is separate from the main data.
- A DBMS gives us the right tools – constraints, triggers, functions, views, transactions and indexes – to enforce these rules at the data layer rather than relying on user discipline.

1.3 Objectives

- Design a normalised relational schema for cases, evidence, categories, users, locations, persons, forensic tests and custody transfers.
- Enforce data integrity using primary keys, foreign keys, unique, check and enumeration constraints.
- Automate chain-of-custody state changes with a trigger and verify chain continuity with a stored function.
- Provide reporting through a view and efficient lookups through indexes.
- Record sensitive activity in a separate MongoDB audit log.
- Build a secure, easy-to-use web interface with login, logout and role-based access.

1.4 Scope and limitations

The system covers evidence registration, custody transfer logging, forensic test recording, chain verification, case listing, audit viewing and user listing. It is designed as an academic prototype: forensic report files are stored as path references rather than uploaded documents, and administrative creation or editing of users and cases is performed through SQL or the seed script rather than through the interface. These items are listed as future enhancements in Chapter 9.

1.5 Technologies used

Table 1.1: Technology stack

Layer	Technology	Purpose
Relational database	MySQL 8	System of record: tables, constraints, trigger, function, view, indexes, transactions
Document database	MongoDB 6	Append-only audit_events collection
Front end	Next.js 16 (App Router), React 19, TypeScript	User interface and server-rendered pages
Back end / API	Next.js route handlers	REST-style endpoints under /api
Authentication	bcryptjs, jose (JWT)	Password hashing and signed, HTTP-only session cookie
Validation	Zod	Server-side validation of every request body
DB drivers	mysql2, mongodb	Connection pooling and queries
Tools	MySQL Workbench / CLI, MongoDB Compass, VS Code	Development and testing

1.6 Team and responsibilities

Table 1.2: Team member responsibilities

Team member	SRN	Responsibilities
Sindhu GV	PES2UG24CS661	Database design, DDL, DML, constraints, advanced SQL, seed data and database testing
Vanishree	PES2UG24CS665	Next.js front end, API integration, authentication and role authorisation, audit integration, documentation and application testing

2. PROBLEM DEFINITION

2.1 Problem statement

To design and implement a database-backed system that records evidence collected for criminal cases and maintains a complete, verifiable, auditable chain of custody for each item from collection to release, while supporting forensic test tracking and case investigation reporting for different classes of users.

2.2 Limitations of the existing (manual) approach

- Paper registers can be lost, damaged or altered, and entries cannot be validated when they are written.
- There is no automatic link between a custody transfer and the current status of the evidence.
- Searching across cases, categories, people or dates is slow and error-prone.
- There is no reliable record of who viewed, created or deleted a record.
- Access is not restricted by role, so anyone with the register can change it.

2.3 Proposed solution

ChainSecure stores all records in a relational schema with enforced constraints. Each custody transfer is numbered per evidence item and inserted inside a transaction; a trigger then marks the evidence as In_Transit and clears its Chain_Verified flag. An authorised officer can later run the chain verification, which calls a MySQL stored function that checks receipt times and the continuity of transfer numbers before the chain is marked verified again. Every important action is written to a separate MongoDB audit collection.

2.4 Functional requirements

- Secure login and logout for registered, active users.
- Register new evidence against a case and a category, with a SHA-256/SHA-512 hash value and storage location.
- Log custody transfers between users and locations with automatic transfer numbering.
- Record forensic tests (type, date, result, report path) against evidence.
- Verify the chain of custody of an evidence item using the stored function.
- List cases with the number of evidence items, and view evidence, transfers and tests.
- Search and filter records on screen; delete evidence with confirmation.
- View the audit log and the list of users (Admin only).

2.5 Non-functional requirements

- **Integrity:** foreign keys, unique keys, check constraints and enumerations must prevent invalid data.
- **Security:** hashed passwords, signed HTTP-only session cookies, server-side authorisation and validation, no secrets in source code.
- **Auditability:** key actions are stored in an append-only log.
- **Usability:** responsive interface, dropdowns instead of raw IDs, clear error messages.
- **Performance:** indexes on frequently filtered columns (status, transfer date).

2.6 User roles

Table 2.1: User roles and permissions

Capability	Admin	Officer	Forensic Officer
Log in / log out	Yes	Yes	Yes
View overview, cases, evidence, custody chain, tests	Yes	Yes	Yes
Register evidence	Yes	Yes	No
Log custody transfer	Yes	Yes	No
Verify chain / delete evidence	Yes	Yes	No
Add forensic test	Yes	No	Yes
View audit log and users	Yes	No	No

3. ER MODEL

3.1 Entities and attributes

The conceptual design identifies eight entity types. CUSTODY_TRANSFER is a weak entity because a transfer has no meaning without the evidence it belongs to; it is identified by the owner's key together with the partial key Transfer_No.

Table 3.1: Entities and their attributes (PK = primary key, FK = foreign key)

Entity	Attributes
USER	User_ID (PK), Name, Role, Department, Phone_Number, Email, Password_Hash, Status, Location_ID (FK)
LOCATION	Location_ID (PK), Location_Name, Location_Type, Address, City, State, Country
CASE	Case_ID (PK), Case_Number, Title, Description, Case_Type, Opened_Date, Status
PERSON	Person_ID (PK), Name, Person_Type, Contact_Info, Date_of_Birth, Identification_No
EVIDENCE_CATEGORY	Category_ID (PK), Category_Name, Category_Description
EVIDENCE	Evidence_ID (PK), Case_ID (FK), Category_ID (FK), Description, Collected_Date, Collected_By (FK), Current_Status, Storage_Location, Hash_Value, Hash_Algorithm, Chain_Verified
FORENSIC_TEST	Test_ID (PK), Evidence_ID (FK), Test_Type, Conducted_By (FK), Test_Date, Result, Report_Path
CUSTODY_TRANSFER (weak)	Transfer_No (partial key), Evidence_ID (owner key), From_User_ID (FK), To_User_ID (FK), Transfer_DateTime, Received_Time, From_Location_ID (FK), Remarks

3.2 Relationships and cardinality

Table 3.2: Relationships between entities

Relationship	Entities	Cardinality	Description
HAS	CASE – EVIDENCE	1 : N	A case has many evidence items; each item belongs to one case
CATEGORISED AS	EVIDENCE_CATEGORY – EVIDENCE	1 : N	Each evidence item has exactly one category
COLLECTED BY	USER – EVIDENCE	1 : N	A user collects many items; each item has one collector
UNDERGOES	EVIDENCE – FORENSIC_TEST	1 : N	An item can undergo many forensic tests
CONDUCTS	USER – FORENSIC_TEST	1 : N	A user conducts many tests
HAS CUSTODY TRANSFER (identifying)	EVIDENCE – CUSTODY_TRANSFER	1 : N	An item has an ordered series of transfers
TRANSFERRED FROM / TO	USER – CUSTODY_TRANSFER	1 : N (twice)	Each transfer has one giver and one receiver

Relationship	Entities	Cardinality	Description
FROM LOCATION	LOCATION – CUSTODY_TRANSFER	1 : N	Each transfer starts at one location
ASSIGNED TO	LOCATION – USER	1 : N	A user is posted at one location
INVOLVES	CASE – PERSON	M : N	People (witness, suspect, victim) are involved in many cases
BELONGS TO	PERSON – EVIDENCE	M : N	Evidence may be linked to several people

3.3 ER diagram

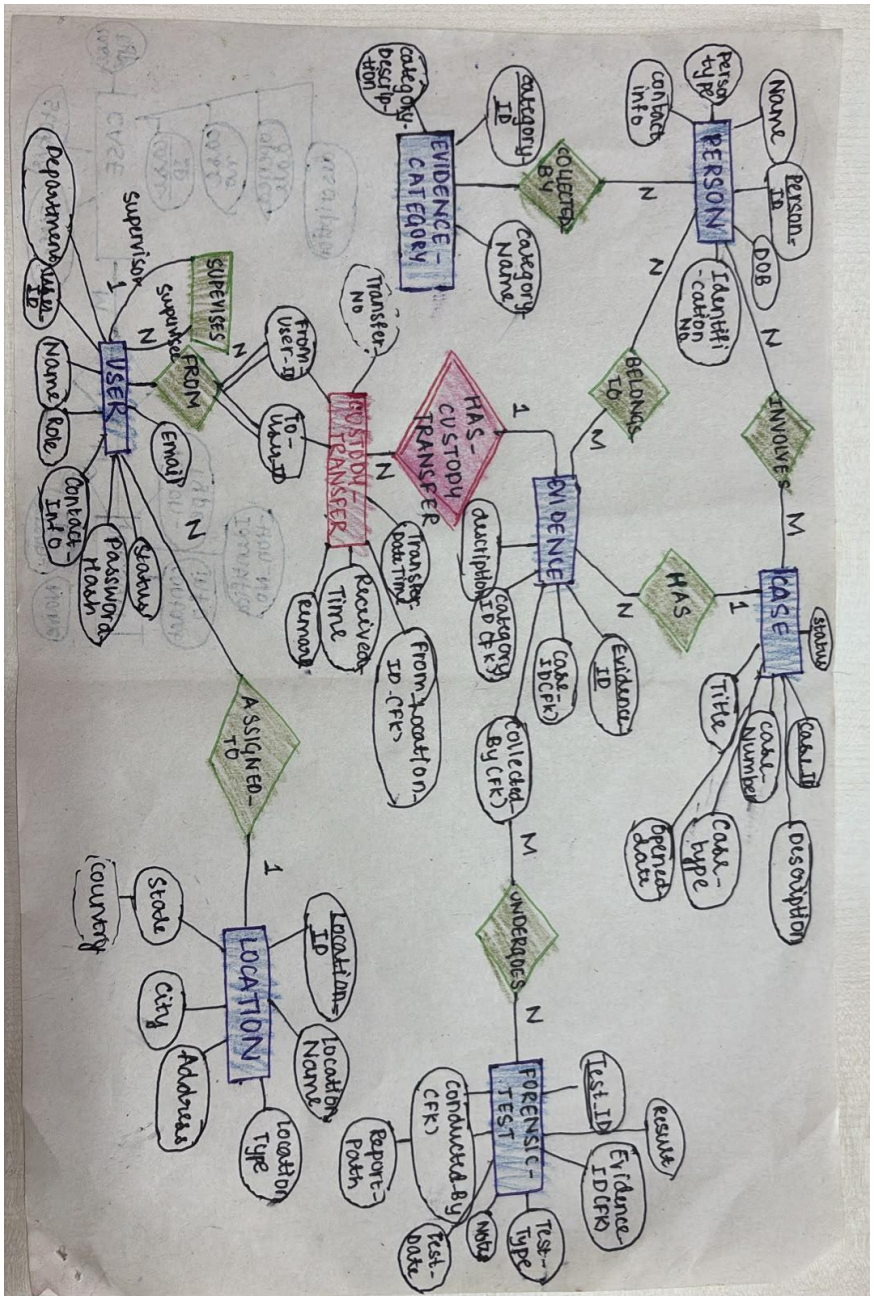


Figure 3.1: Entity–Relationship diagram of ChainSecure

3.4 Design notes

- Evidence status is restricted to Collected, Stored, In_Lab, In_Transit and Released; Chain_Verified is a Boolean flag that the trigger and the verification function maintain.
- Users have exactly one role (Admin, Officer or Forensic Officer) and an Active/Inactive status that controls login.
- Hash_Value and Hash_Algorithm are stored with every item so that its integrity can be re-checked later.

4. ER TO RELATIONAL MAPPING

4.1 Steps of the algorithm for the chosen problem

Table 4.1: Application of the ER-to-relational mapping algorithm

Step	Rule	Applied to ChainSecure
1	Map regular entity types	USER, LOCATION, CASE, PERSON, EVIDENCE_CATEGORY, EVIDENCE and FORENSIC_TEST become relations; simple attributes become columns and the key attribute becomes the primary key.
2	Map weak entity types	CUSTODY_TRANSFER becomes a relation containing the owner's primary key (Evidence_ID) and the partial key (Transfer_No). A surrogate Transfer_ID is the primary key and (Evidence_ID, Transfer_No) is declared UNIQUE, with ON DELETE CASCADE from EVIDENCE.
3	Map binary 1:1 relationships	None in this design.
4	Map binary 1:N relationships	The foreign key is placed on the N side: EVIDENCE(Case_ID, Category_ID, Collected_By), FORENSIC_TEST(Evidence_ID, Conducted_By), CUSTODY_TRANSFER(From_User_ID, To_User_ID, From_Location_ID), USER(Location_ID).
5	Map binary M:N relationships	INVOLVES(Case_ID, Person_ID) and BELONGS_TO(Person_ID, Evidence_ID) become separate relations whose primary key is the combination of the two foreign keys.
6	Map multivalued attributes	None in this design.
7	Map n-ary relationships	None in this design.

4.2 Complete diagram of relational mapping

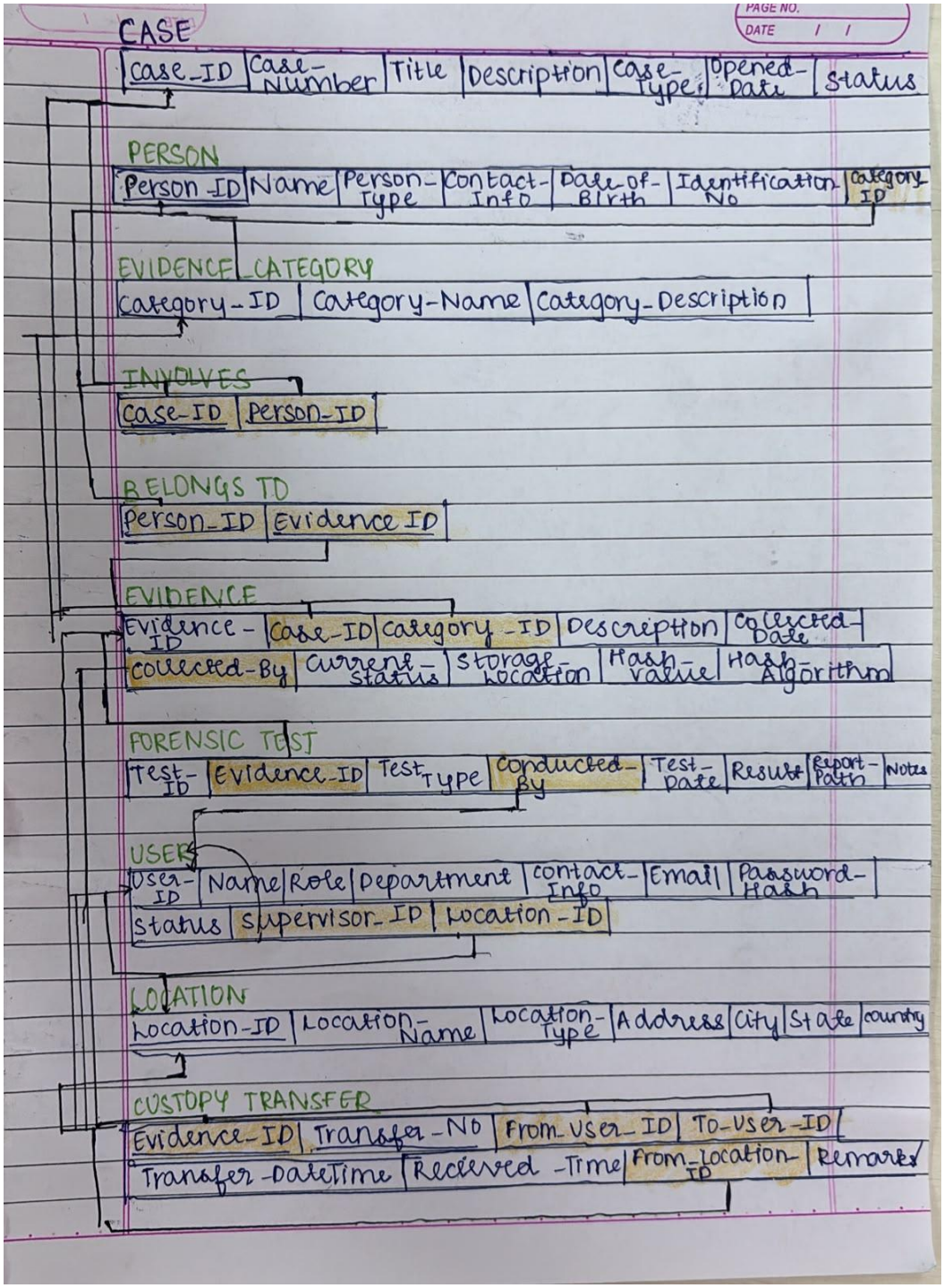


Figure 4.1: Relational schema of ChainSecure

4.3 Resulting relations

Table 4.2: Relations, primary keys and foreign keys

Relation	Primary key	Foreign keys / unique keys
evidence_category	Category_ID	UNIQUE (Category_Name)
location	Location_ID	–
user	User_ID	Location_ID → location; UNIQUE (Phone_Number), UNIQUE (Email)
case	Case_ID	UNIQUE (Case_Number)
person	Person_ID	UNIQUE (Identification_No)
involves	(Case_ID, Person_ID)	Case_ID → case; Person_ID → person (both ON DELETE CASCADE)
evidence	Evidence_ID	Case_ID → case; Category_ID → evidence_category; Collected_By → user
belongs_to	(Person_ID, Evidence_ID)	Person_ID → person; Evidence_ID → evidence (both ON DELETE CASCADE)
forensic_test	Test_ID	Evidence_ID → evidence (ON DELETE CASCADE); Conducted_By → user
custody_transfer	Transfer_ID	Evidence_ID → evidence (CASCADE); From_User_ID, To_User_ID → user; From_Location_ID → location; UNIQUE (Evidence_ID, Transfer_No)

4.4 Normalisation

All relations are in First Normal Form because every column holds a single atomic value. Every non-key attribute depends on the whole of its relation's key (Second Normal Form), and there are no transitive dependencies between non-key attributes (Third Normal Form): descriptive data about categories, locations, cases and users is stored once in its own relation and referenced by foreign key from EVIDENCE, CUSTODY_TRANSFER and FORENSIC_TEST.

5. DDL STATEMENTS

The schema is created in the database **legal_evidence_custody_db**. Each statement below is taken from schema.sql. After running it, capture the table structure with DESCRIBE <table_name>; and paste the screenshot in the box provided.

5.1 Database creation

```
CREATE DATABASE IF NOT EXISTS legal_evidence_custody_db;
USE legal_evidence_custody_db;
```

```
mysql> CREATE DATABASE legal_evidence_custody_db;
Query OK, 1 row affected (0.01 sec)

mysql> USE legal_evidence_custody_db;
Database changed
mysql>
```

Figure 5.1: Database creation and SHOW TABLES output

5.2 EVIDENCE_CATEGORY table

```
CREATE TABLE IF NOT EXISTS evidence_category (
  Category_ID INT AUTO_INCREMENT PRIMARY KEY,
  Category_Name VARCHAR(80) NOT NULL UNIQUE,
  Category_Description VARCHAR(255) NOT NULL
);
```

```
mysql> CREATE TABLE IF NOT EXISTS evidence_category (
    ->   Category_ID INT AUTO_INCREMENT PRIMARY KEY,
    ->   Category_Name VARCHAR(80) NOT NULL UNIQUE,
    ->   Category_Description VARCHAR(255) NOT NULL
    -> );
Query OK, 0 rows affected (0.09 sec)

mysql> DESCRIBE evidence_category;
+----------------------+--------------+------+-----+---------+----------------+
| Field                | Type         | Null | Key | Default | Extra          |
+----------------------+--------------+------+-----+---------+----------------+
| Category_ID          | int          | NO   | PRI | NULL    | auto_increment |
| Category_Name        | varchar(80)  | NO   | UNI | NULL    |                |
| Category_Description | varchar(255) | NO   |     | NULL    |                |
+----------------------+--------------+------+-----+---------+----------------+
3 rows in set (0.02 sec)
```

Figure 5.2: DDL of evidence_category table

5.3 LOCATION table

```
CREATE TABLE IF NOT EXISTS location (
  Location_ID INT AUTO_INCREMENT PRIMARY KEY,
  Location_Name VARCHAR(120) NOT NULL,
  Location_Type VARCHAR(80) NOT NULL,
  Address VARCHAR(255) NOT NULL,
  City VARCHAR(80) NOT NULL,
  State VARCHAR(80) NOT NULL,
  Country VARCHAR(80) NOT NULL
);
```

```
mysql> CREATE TABLE IF NOT EXISTS location (
    ->   Location_ID INT AUTO_INCREMENT PRIMARY KEY,
    ->   Location_Name VARCHAR(120) NOT NULL,
    ->   Location_Type VARCHAR(80) NOT NULL,
    ->   Address VARCHAR(255) NOT NULL,
    ->   City VARCHAR(80) NOT NULL,
    ->   State VARCHAR(80) NOT NULL,
    ->   Country VARCHAR(80) NOT NULL
    -> );
Query OK, 0 rows affected (0.12 sec)

mysql> DESCRIBE location;
+---------------+-------------+------+-----+---------+----------------+
| Field         | Type        | Null | Key | Default | Extra          |
+---------------+-------------+------+-----+---------+----------------+
| Location_ID   | int         | NO   | PRI | NULL    | auto_increment |
| Location_Name | varchar(120)| NO   |     | NULL    |                |
| Location_Type | varchar(80) | NO   |     | NULL    |                |
| Address       | varchar(255)| NO   |     | NULL    |                |
| City          | varchar(80) | NO   |     | NULL    |                |
| State         | varchar(80) | NO   |     | NULL    |                |
| Country       | varchar(80) | NO   |     | NULL    |                |
+---------------+-------------+------+-----+---------+----------------+
7 rows in set (0.01 sec)

mysql>
```

Figure 5.3: DDL of location table

5.4 USER table

```
CREATE TABLE IF NOT EXISTS `user` (
  User_ID INT AUTO_INCREMENT PRIMARY KEY,
  Name VARCHAR(120) NOT NULL,
  Role ENUM('Admin','Officer','Forensic Officer') NOT NULL,
  Department VARCHAR(120) NOT NULL,
  Phone_Number VARCHAR(20) NOT NULL UNIQUE,
  Email VARCHAR(160) NOT NULL UNIQUE,
  Password_Hash VARCHAR(255) NOT NULL,
  Status ENUM('Active','Inactive') NOT NULL DEFAULT 'Active',
  Location_ID INT NULL,
  CONSTRAINT fk_user_location FOREIGN KEY (Location_ID) REFERENCES
location(Location_ID)
);
```

```
mysql> CREATE TABLE IF NOT EXISTS `user` (
    ->   User_ID INT AUTO_INCREMENT PRIMARY KEY,
    ->   Name VARCHAR(120) NOT NULL,
    ->   Role ENUM('Admin','Officer','Forensic Officer') NOT NULL,
    ->   Department VARCHAR(120) NOT NULL,
    ->   Phone_Number VARCHAR(20) NOT NULL UNIQUE,
    ->   Email VARCHAR(160) NOT NULL UNIQUE,
    ->   Password_Hash VARCHAR(255) NOT NULL,
    ->   Status ENUM('Active','Inactive') NOT NULL DEFAULT 'Active',
    ->   Location_ID INT NULL,
    ->   CONSTRAINT fk_user_location FOREIGN KEY (Location_ID) REFERENCES location(Location_ID)
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql> DESCRIBE `user`;
+---------------+-----------------------------------------+------+-----+---------+----------------+
| Field         | Type                                    | Null | Key | Default | Extra          |
+---------------+-----------------------------------------+------+-----+---------+----------------+
| User_ID       | int                                     | NO   | PRI | NULL    | auto_increment |
| Name          | varchar(120)                            | NO   |     | NULL    |                |
| Role          | enum('Admin','Officer','Forensic Officer') | NO   |     | NULL    |                |
| Department    | varchar(120)                            | NO   |     | NULL    |                |
| Phone_Number  | varchar(20)                             | NO   | UNI | NULL    |                |
| Email         | varchar(160)                            | NO   | UNI | NULL    |                |
| Password_Hash | varchar(255)                            | NO   |     | NULL    |                |
| Status        | enum('Active','Inactive')               | NO   |     | Active  |                |
| Location_ID   | int                                     | YES  | MUL | NULL    |                |
+---------------+-----------------------------------------+------+-----+---------+----------------+
9 rows in set (0.03 sec)

mysql>
```

Figure 5.4: DDL of user table

5.5 CASE table

```
CREATE TABLE IF NOT EXISTS `case` (
  Case_ID INT AUTO_INCREMENT PRIMARY KEY,
  Case_Number VARCHAR(40) NOT NULL UNIQUE,
  Title VARCHAR(180) NOT NULL,
  Description TEXT NOT NULL,
  Case_Type VARCHAR(80) NOT NULL,
  Opened_Date DATE NOT NULL,
  Status ENUM('Open','Under Investigation','Closed') NOT NULL,
  CHECK (Opened_Date >= '2000-01-01')
);
```

```
mysql> CREATE TABLE IF NOT EXISTS `case` (
    ->   Case_ID INT AUTO_INCREMENT PRIMARY KEY,
    ->   Case_Number VARCHAR(40) NOT NULL UNIQUE,
    ->   Title VARCHAR(180) NOT NULL,
    ->   Description TEXT NOT NULL,
    ->   Case_Type VARCHAR(80) NOT NULL,
    ->   Opened_Date DATE NOT NULL,
    ->   Status ENUM('Open','Under Investigation','Closed') NOT NULL,
    ->   CHECK (Opened_Date >= '2000-01-01')
    -> );
Query OK, 0 rows affected (0.10 sec)

mysql> DESCRIBE `case`;
+-------------+-----------------------------------------+------+-----+---------+----------------+
| Field       | Type                                    | Null | Key | Default | Extra          |
+-------------+-----------------------------------------+------+-----+---------+----------------+
| Case_ID     | int                                     | NO   | PRI | NULL    | auto_increment |
| Case_Number | varchar(40)                             | NO   | UNI | NULL    |                |
| Title       | varchar(180)                            | NO   |     | NULL    |                |
| Description | text                                    | NO   |     | NULL    |                |
| Case_Type   | varchar(80)                             | NO   |     | NULL    |                |
| Opened_Date | date                                    | NO   |     | NULL    |                |
| Status      | enum('Open','Under Investigation','Closed') | NO   |     | NULL    |                |
+-------------+-----------------------------------------+------+-----+---------+----------------+
7 rows in set (0.01 sec)

mysql>
```

Figure 5.5: DDL of case table

5.6 PERSON table

```
CREATE TABLE IF NOT EXISTS person (
  Person_ID INT AUTO_INCREMENT PRIMARY KEY,
  Name VARCHAR(120) NOT NULL,
  Person_Type ENUM('Witness','Suspect','Victim') NOT NULL,
  Contact_Info VARCHAR(160) NOT NULL,
  Date_of_Birth DATE NOT NULL,
  Identification_No VARCHAR(80) NOT NULL UNIQUE
);
```

```
mysql> CREATE TABLE IF NOT EXISTS person (
    ->   Person_ID INT AUTO_INCREMENT PRIMARY KEY,
    ->   Name VARCHAR(120) NOT NULL,
    ->   Person_Type ENUM('Witness','Suspect','Victim') NOT NULL,
    ->   Contact_Info VARCHAR(160) NOT NULL,
    ->   Date_of_Birth DATE NOT NULL,
    ->   Identification_No VARCHAR(80) NOT NULL UNIQUE
    -> );
Query OK, 0 rows affected (0.11 sec)

mysql> DESCRIBE person;
+-----------------+-----------------------------------+------+-----+---------+----------------+
| Field           | Type                              | Null | Key | Default | Extra          |
+-----------------+-----------------------------------+------+-----+---------+----------------+
| Person_ID       | int                               | NO   | PRI | NULL    | auto_increment |
| Name            | varchar(120)                      | NO   |     | NULL    |                |
| Person_Type     | enum('Witness','Suspect','Victim') | NO   |     | NULL    |                |
| Contact_Info    | varchar(160)                      | NO   |     | NULL    |                |
| Date_of_Birth   | date                              | NO   |     | NULL    |                |
| Identification_No | varchar(80)                     | NO   | UNI | NULL    |                |
+-----------------+-----------------------------------+------+-----+---------+----------------+
6 rows in set (0.00 sec)

mysql>
```

Figure 5.6: DDL of person table

5.7 INVOLVES table

```
CREATE TABLE IF NOT EXISTS involves (
  Case_ID INT NOT NULL,
  Person_ID INT NOT NULL,
  PRIMARY KEY (Case_ID, Person_ID),
  FOREIGN KEY (Case_ID) REFERENCES `case`(Case_ID) ON DELETE CASCADE,
  FOREIGN KEY (Person_ID) REFERENCES person(Person_ID) ON DELETE CASCADE
);
```

```
mysql> CREATE TABLE IF NOT EXISTS involves (
    ->   Case_ID INT NOT NULL,
    ->   Person_ID INT NOT NULL,
    ->   PRIMARY KEY (Case_ID, Person_ID),
    ->   FOREIGN KEY (Case_ID) REFERENCES `case`(Case_ID) ON DELETE CASCADE,
    ->   FOREIGN KEY (Person_ID) REFERENCES person(Person_ID) ON DELETE CASCADE
    -> );
Query OK, 0 rows affected (0.06 sec)

mysql> DESCRIBE involves;
+-----------+------+------+-----+---------+-------+
| Field     | Type | Null | Key | Default | Extra |
+-----------+------+------+-----+---------+-------+
| Case_ID   | int  | NO   | PRI | NULL    |       |
| Person_ID | int  | NO   | PRI | NULL    |       |
+-----------+------+------+-----+---------+-------+
2 rows in set (0.00 sec)

mysql>
```

Figure 5.7: DDL of involves table

5.8 EVIDENCE table

```
CREATE TABLE IF NOT EXISTS evidence (
  Evidence_ID INT AUTO_INCREMENT PRIMARY KEY,
  Case_ID INT NOT NULL,
  Category_ID INT NOT NULL,
  Description VARCHAR(500) NOT NULL,
  Collected_Date DATE NOT NULL,
  Collected_By INT NOT NULL,
  Current_Status ENUM('Collected','Stored','In_Lab','In_Transit','Released') NOT NULL,
  Storage_Location VARCHAR(160) NOT NULL,
  Hash_Value CHAR(64) NOT NULL,
  Hash_Algorithm ENUM('SHA256','SHA512') NOT NULL DEFAULT 'SHA256',
  Chain_Verified BOOLEAN NOT NULL DEFAULT TRUE,
  FOREIGN KEY (Case_ID) REFERENCES `case`(Case_ID),
  FOREIGN KEY (Category_ID) REFERENCES evidence_category(Category_ID),
  FOREIGN KEY (Collected_By) REFERENCES `user`(User_ID),
  INDEX idx_evidence_status (Current_Status),
  INDEX idx_evidence_collected_date (Collected_Date)
);
```

```
mysql> CREATE TABLE IF NOT EXISTS evidence (
    ->   Evidence_ID INT AUTO_INCREMENT PRIMARY KEY,
    ->   Case_ID INT NOT NULL,
    ->   Category_ID INT NOT NULL,
    ->   Description VARCHAR(500) NOT NULL,
    ->   Collected_Date DATE NOT NULL,
    ->   Collected_By INT NOT NULL,
    ->   Current_Status ENUM('Collected','Stored','In_Lab','In_Transit','Released') NOT NULL,
    ->   Storage_Location VARCHAR(160) NOT NULL,
    ->   Hash_Value CHAR(64) NOT NULL,
    ->   Hash_Algorithm ENUM('SHA256','SHA512') NOT NULL DEFAULT 'SHA256',
    ->   Chain_Verified BOOLEAN NOT NULL DEFAULT TRUE,
    ->   FOREIGN KEY (Case_ID) REFERENCES `case`(Case_ID),
    ->   FOREIGN KEY (Category_ID) REFERENCES evidence_category(Category_ID),
    ->   FOREIGN KEY (Collected_By) REFERENCES `user`(User_ID),
    ->   INDEX idx_evidence_status (Current_Status),
    ->   INDEX idx_evidence_collected_date (Collected_Date)
    -> );
Query OK, 0 rows affected (0.08 sec)

mysql> DESCRIBE evidence;
+-----------------+-----------------------------------------------------------------+------+-----+---------+----------------+
| Field           | Type                                                            | Null | Key | Default | Extra          |
+-----------------+-----------------------------------------------------------------+------+-----+---------+----------------+
| Evidence_ID     | int                                                             | NO   | PRI | NULL    | auto_increment |
| Case_ID         | int                                                             | NO   | MUL | NULL    |                |
| Category_ID     | int                                                             | NO   | MUL | NULL    |                |
| Description     | varchar(500)                                                    | NO   |     | NULL    |                |
| Collected_Date  | date                                                            | NO   | MUL | NULL    |                |
| Collected_By    | int                                                             | NO   | MUL | NULL    |                |
| Current_Status  | enum('Collected','Stored','In_Lab','In_Transit','Released')     | NO   | MUL | NULL    |                |
| Storage_Location | varchar(160)                                                   | NO   |     | NULL    |                |
| Hash_Value      | char(64)                                                        | NO   |     | NULL    |                |
| Hash_Algorithm  | enum('SHA256','SHA512')                                         | NO   |     | SHA256  |                |
| Chain_Verified  | tinyint(1)                                                      | NO   |     | 1       |                |
+-----------------+-----------------------------------------------------------------+------+-----+---------+----------------+
11 rows in set (0.03 sec)
```

Figure 5.8: DDL of evidence table

5.9 BELONGS_TO table

```
CREATE TABLE IF NOT EXISTS belongs_to (
  Person_ID INT NOT NULL,
  Evidence_ID INT NOT NULL,
  PRIMARY KEY (Person_ID, Evidence_ID),
  FOREIGN KEY (Person_ID) REFERENCES person(Person_ID) ON DELETE CASCADE,
  FOREIGN KEY (Evidence_ID) REFERENCES evidence(Evidence_ID) ON DELETE CASCADE
);
```

```
mysql> CREATE TABLE IF NOT EXISTS belongs_to (
    ->   Person_ID INT NOT NULL,
    ->   Evidence_ID INT NOT NULL,
    ->   PRIMARY KEY (Person_ID, Evidence_ID),
    ->   FOREIGN KEY (Person_ID) REFERENCES person(Person_ID) ON DELETE CASCADE,
    ->   FOREIGN KEY (Evidence_ID) REFERENCES evidence(Evidence_ID) ON DELETE CASCADE
    -> );
Query OK, 0 rows affected (0.09 sec)

mysql> DESCRIBE belongs_to;
+-------------+------+------+-----+---------+-------+
| Field       | Type | Null | Key | Default | Extra |
+-------------+------+------+-----+---------+-------+
| Person_ID   | int  | NO   | PRI | NULL    |       |
| Evidence_ID | int  | NO   | PRI | NULL    |       |
+-------------+------+------+-----+---------+-------+
2 rows in set (0.00 sec)

mysql>
```

Figure 5.9: DDL of belongs_to table

5.10 FORENSIC_TEST table

```
CREATE TABLE IF NOT EXISTS forensic_test (
  Test_ID INT AUTO_INCREMENT PRIMARY KEY,
  Evidence_ID INT NOT NULL,
  Test_Type VARCHAR(160) NOT NULL,
  Conducted_By INT NOT NULL,
  Test_Date DATE NOT NULL,
  Result TEXT NOT NULL,
  Report_Path VARCHAR(255) NOT NULL,
  FOREIGN KEY (Evidence_ID) REFERENCES evidence(Evidence_ID) ON DELETE CASCADE,
  FOREIGN KEY (Conducted_By) REFERENCES `user`(User_ID)
);
```

```
mysql> CREATE TABLE IF NOT EXISTS forensic_test (
    ->   Test_ID INT AUTO_INCREMENT PRIMARY KEY,
    ->   Evidence_ID INT NOT NULL,
    ->   Test_Type VARCHAR(160) NOT NULL,
    ->   Conducted_By INT NOT NULL,
    ->   Test_Date DATE NOT NULL,
    ->   Result TEXT NOT NULL,
    ->   Report_Path VARCHAR(255) NOT NULL,
    ->   FOREIGN KEY (Evidence_ID) REFERENCES evidence(Evidence_ID) ON DELETE CASCADE,
    ->   FOREIGN KEY (Conducted_By) REFERENCES `user`(User_ID)
    -> );
Query OK, 0 rows affected (0.06 sec)

mysql> DESCRIBE forensic_test;
+--------------+--------------+------+-----+---------+----------------+
| Field        | Type         | Null | Key | Default | Extra          |
+--------------+--------------+------+-----+---------+----------------+
| Test_ID      | int          | NO   | PRI | NULL    | auto_increment |
| Evidence_ID  | int          | NO   | MUL | NULL    |                |
| Test_Type    | varchar(160) | NO   |     | NULL    |                |
| Conducted_By | int          | NO   | MUL | NULL    |                |
| Test_Date    | date         | NO   |     | NULL    |                |
| Result       | text         | NO   |     | NULL    |                |
| Report_Path  | varchar(255) | NO   |     | NULL    |                |
+--------------+--------------+------+-----+---------+----------------+
7 rows in set (0.00 sec)

mysql>
```

Figure 5.10: DDL of forensic_test table

5.11 CUSTODY_TRANSFER table

```
CREATE TABLE IF NOT EXISTS custody_transfer (
  Transfer_ID INT AUTO_INCREMENT PRIMARY KEY,
  Evidence_ID INT NOT NULL,
  Transfer_No INT NOT NULL,
  From_User_ID INT NOT NULL,
  To_User_ID INT NOT NULL,
  Transfer_DateTime DATETIME NOT NULL,
  Received_Time DATETIME NULL,
  From_Location_ID INT NOT NULL,
  Remarks VARCHAR(500) NOT NULL,
  UNIQUE KEY uq_evidence_transfer (Evidence_ID, Transfer_No),
  FOREIGN KEY (Evidence_ID) REFERENCES evidence(Evidence_ID) ON DELETE CASCADE,
  FOREIGN KEY (From_User_ID) REFERENCES `user`(User_ID),
  FOREIGN KEY (To_User_ID) REFERENCES `user`(User_ID),
  FOREIGN KEY (From_Location_ID) REFERENCES location(Location_ID),
  CHECK (Received_Time IS NULL OR Received_Time >= Transfer_DateTime),
  INDEX idx_transfer_datetime (Transfer_DateTime)
);
```

```
mysql> CREATE TABLE IF NOT EXISTS custody_transfer (
    ->   Transfer_ID INT AUTO_INCREMENT PRIMARY KEY,
    ->   Evidence_ID INT NOT NULL,
    ->   Transfer_No INT NOT NULL,
    ->   From_User_ID INT NOT NULL,
    ->   To_User_ID INT NOT NULL,
    ->   Transfer_DateTime DATETIME NOT NULL,
    ->   Received_Time DATETIME NULL,
    ->   From_Location_ID INT NOT NULL,
    ->   Remarks VARCHAR(500) NOT NULL,
    ->   UNIQUE KEY uq_evidence_transfer (Evidence_ID, Transfer_No),
    ->   FOREIGN KEY (Evidence_ID) REFERENCES evidence(Evidence_ID) ON DELETE CASCADE,
    ->   FOREIGN KEY (From_User_ID) REFERENCES `user`(User_ID),
    ->   FOREIGN KEY (To_User_ID) REFERENCES `user`(User_ID),
    ->   FOREIGN KEY (From_Location_ID) REFERENCES location(Location_ID),
    ->   CHECK (Received_Time IS NULL OR Received_Time >= Transfer_DateTime),
    ->   INDEX idx_transfer_datetime (Transfer_DateTime)
    -> );
Query OK, 0 rows affected (0.14 sec)

mysql> DESCRIBE custody_transfer;
+-------------------+--------------+------+-----+---------+----------------+
| Field             | Type         | Null | Key | Default | Extra          |
+-------------------+--------------+------+-----+---------+----------------+
| Transfer_ID       | int          | NO   | PRI | NULL    | auto_increment |
| Evidence_ID       | int          | NO   | MUL | NULL    |                |
| Transfer_No       | int          | NO   |     | NULL    |                |
| From_User_ID      | int          | NO   | MUL | NULL    |                |
| To_User_ID        | int          | NO   | MUL | NULL    |                |
| Transfer_DateTime | datetime     | NO   | MUL | NULL    |                |
| Received_Time     | datetime     | YES  |     | NULL    |                |
| From_Location_ID  | int          | NO   | MUL | NULL    |                |
| Remarks           | varchar(500) | NO   |     | NULL    |                |
+-------------------+--------------+------+-----+---------+----------------+
9 rows in set (0.01 sec)
```

Figure 5.11: DDL of custody_transfer table

5.12 Summary of constraints

Table 5.1: Constraints used in the schema

Constraint type	Where used
PRIMARY KEY	Every relation; composite keys in involves and belongs_to
FOREIGN KEY	evidence, forensic_test, custody_transfer, user, involves, belongs_to

Constraint type	Where used
UNIQUE	Category_Name, Case_Number, Email, Phone_Number, Identification_No, (Evidence_ID, Transfer_No)
NOT NULL	All mandatory attributes
ENUM	Role, Status, Person_Type, Case Status, Current_Status, Hash_Algorithm
CHECK	Opened_Date ≥ 2000-01-01; Received_Time ≥ Transfer_DateTime
DEFAULT	Chain_Verified = TRUE, Hash_Algorithm = SHA256, user Status = Active
INDEX	idx_evidence_status, idx_evidence_collected_date, idx_transfer_datetime

6. DML STATEMENTS

Sample data was inserted into every table. For each INSERT, run SELECT * FROM <table>; and paste the output below to show the inserted values. The same data is also loaded by the application's seed script, which stores real bcrypt password hashes for the demo users.

6.1 EVIDENCE_CATEGORY

```
INSERT INTO evidence_category
(Category_Name, Category_Description)
VALUES
('Digital', 'Digital evidence such as mobile phones and CCTV footage'),
('Physical', 'Physical objects collected from a crime scene'),
('Biological', 'Biological samples collected for forensic analysis'),
('Documentary', 'Documents and written records');
```

```
mysql> use legal_evidence_custody_db;
Database changed
mysql> INSERT INTO evidence_category
    -> (Category_Name, Category_Description)
    -> VALUES
    -> ('Digital', 'Digital evidence such as mobile phones and CCTV footage'),
    -> ('Physical', 'Physical objects collected from a crime scene'),
    -> ('Biological', 'Biological samples collected for forensic analysis'),
    -> ('Documentary', 'Documents and written records');
Query OK, 4 rows affected (0.04 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM evidence_category;
+-------------+---------------+-----------------------------------------------------------+
| Category_ID | Category_Name | Category_Description                                      |
+-------------+---------------+-----------------------------------------------------------+
|           1 | Digital       | Digital evidence such as mobile phones and CCTV footage   |
|           2 | Physical      | Physical objects collected from a crime scene             |
|           3 | Biological    | Biological samples collected for forensic analysis        |
|           4 | Documentary   | Documents and written records                             |
+-------------+---------------+-----------------------------------------------------------+
4 rows in set (0.00 sec)
```

Figure 6.1: Records inserted into evidence_category

6.2 LOCATION

```
INSERT INTO location
(Location_Name, Location_Type, Address, City, State, Country)
VALUES
('Central Police Station', 'Police Station', 'MG Road',
 'Bengaluru', 'Karnataka', 'India'),
('Evidence Storage Room', 'Storage', 'Central Police Station',
 'Bengaluru', 'Karnataka', 'India'),
('Forensic Science Laboratory', 'Forensic Lab', 'Electronic City',
 'Bengaluru', 'Karnataka', 'India'),
('District Court', 'Court', 'Court Road',
 'Bengaluru', 'Karnataka', 'India');
```

```
mysql> INSERT INTO location
    -> (Location_Name, Location_Type, Address, City, State, Country)
    -> VALUES
    -> ('Central Police Station', 'Police Station', 'MG Road',
    ->  'Bengaluru', 'Karnataka', 'India'),
    -> ('Evidence Storage Room', 'Storage', 'Central Police Station',
    ->  'Bengaluru', 'Karnataka', 'India'),
    -> ('Forensic Science Laboratory', 'Forensic Lab', 'Electronic City',
    ->  'Bengaluru', 'Karnataka', 'India'),
    -> ('District Court', 'Court', 'Court Road',
    ->  'Bengaluru', 'Karnataka', 'India');
Query OK, 4 rows affected (0.02 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM location;
+-------------+-----------------------------+---------------+-----------------------+-----------+-----------+---------+
| Location_ID | Location_Name               | Location_Type | Address               | City      | State     | Country |
+-------------+-----------------------------+---------------+-----------------------+-----------+-----------+---------+
|           1 | Central Police Station      | Police Station | MG Road              | Bengaluru | Karnataka | India   |
|           2 | Evidence Storage Room       | Storage       | Central Police Station | Bengaluru | Karnataka | India   |
|           3 | Forensic Science Laboratory | Forensic Lab  | Electronic City       | Bengaluru | Karnataka | India   |
|           4 | District Court              | Court         | Court Road            | Bengaluru | Karnataka | India   |
+-------------+-----------------------------+---------------+-----------------------+-----------+-----------+---------+
4 rows in set (0.00 sec)
```

Figure 6.2: Records inserted into location

6.3 USER

```
INSERT INTO `user`
(Name, Role, Department, Phone_Number, Email,
 Password_Hash, Status, Location_ID)
VALUES
('Ravi Kumar', 'Officer', 'Investigation', '9876543210',
 'ravi@chainsecure.com', 'HASH_RAVI_123', 'Active', 1),
('Priya Sharma', 'Forensic Officer', 'Forensics', '9876543211',
 'priya@chainsecure.com', 'HASH_PRIYA_123', 'Active', 3),
('Arjun Rao', 'Officer', 'Investigation', '9876543212',
 'arjun@chainsecure.com', 'HASH_ARJUN_123', 'Active', 1),
('Meera Singh', 'Admin', 'Administration', '9876543213',
 'meera@chainsecure.com', 'HASH_MEERA_123', 'Active', 1);
```

```
mysql> INSERT INTO `user`
    -> (Name, Role, Department, Phone_Number, Email,
    ->  Password_Hash, Status, Location_ID)
    -> VALUES
    -> ('Ravi Kumar', 'Officer', 'Investigation', '9876543210',
    ->  'ravi@chainsecure.com', 'HASH_RAVI_123', 'Active', 1),
    -> ('Priya Sharma', 'Forensic Officer', 'Forensics', '9876543211',
    ->  'priya@chainsecure.com', 'HASH_PRIYA_123', 'Active', 3),
    -> ('Arjun Rao', 'Officer', 'Investigation', '9876543212',
    ->  'arjun@chainsecure.com', 'HASH_ARJUN_123', 'Active', 1),
    -> ('Meera Singh', 'Admin', 'Administration', '9876543213',
    ->  'meera@chainsecure.com', 'HASH_MEERA_123', 'Active', 1);
Query OK, 4 rows affected (0.03 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM `user`;
+---------+-------------+-----------------+----------------+--------------+-----------------------+---------------+--------+-------------+
| User_ID | Name        | Role            | Department     | Phone_Number | Email                 | Password_Hash | Status | Location_ID |
+---------+-------------+-----------------+----------------+--------------+-----------------------+---------------+--------+-------------+
|       1 | Ravi Kumar  | Officer         | Investigation  | 9876543210   | ravi@chainsecure.com  | HASH_RAVI_123 | Active |           1 |
|       2 | Priya Sharma | Forensic Officer | Forensics     | 9876543211   | priya@chainsecure.com | HASH_PRIYA_123 | Active |          3 |
|       3 | Arjun Rao   | Officer         | Investigation  | 9876543212   | arjun@chainsecure.com | HASH_ARJUN_123 | Active |          1 |
|       4 | Meera Singh | Admin           | Administration | 9876543213   | meera@chainsecure.com | HASH_MEERA_123 | Active |          1 |
+---------+-------------+-----------------+----------------+--------------+-----------------------+---------------+--------+-------------+
4 rows in set (0.00 sec)
```

Figure 6.3: Records inserted into user

6.4 CASE

```
INSERT INTO `case`
(Case_Number, Title, Description, Case_Type, Opened_Date, Status)
VALUES
('CASE-001',
 'Jewellery Store Robbery',
 'Investigation of a robbery at a jewellery store',
 'Robbery', '2026-09-01', 'Open'),

('CASE-002',
 'Vehicle Theft',
 'Investigation of a stolen vehicle',
 'Theft', '2026-09-05', 'Open'),

('CASE-003',
 'Online Financial Fraud',
 'Investigation of an online financial fraud complaint',
 'Cyber Crime', '2026-09-10', 'Under Investigation');
```

```
mysql> INSERT INTO `case`
    -> (Case_Number, Title, Description, Case_Type, Opened_Date, Status)
    -> VALUES
    -> ('CASE-001',
    ->  'Jewellery Store Robbery',
    ->  'Investigation of a robbery at a jewellery store',
    ->  'Robbery', '2026-09-01', 'Open'),
    -> ('CASE-002',
    ->  'Vehicle Theft',
    ->  'Investigation of a stolen vehicle',
    ->  'Theft', '2026-09-05', 'Open'),
    -> ('CASE-003',
    ->  'Online Financial Fraud',
    ->  'Investigation of an online financial fraud complaint',
    ->  'Cyber Crime', '2026-09-10', 'Under Investigation');
Query OK, 3 rows affected (0.03 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM `case`;
+---------+-------------+-------------------------+-----------------------------------------------------+-------------+-------------+----------------------+
| Case_ID | Case_Number | Title                   | Description                                         | Case_Type   | Opened_Date | Status               |
+---------+-------------+-------------------------+-----------------------------------------------------+-------------+-------------+----------------------+
|       1 | CASE-001    | Jewellery Store Robbery | Investigation of a robbery at a jewellery store     | Robbery     | 2026-09-01  | Open                 |
|       2 | CASE-002    | Vehicle Theft           | Investigation of a stolen vehicle                   | Theft       | 2026-09-05  | Open                 |
|       3 | CASE-003    | Online Financial Fraud  | Investigation of an online financial fraud complaint | Cyber Crime | 2026-09-10  | Under Investigation  |
+---------+-------------+-------------------------+-----------------------------------------------------+-------------+-------------+----------------------+
3 rows in set (0.00 sec)

mysql>
```

Figure 6.4: Records inserted into case

6.5 PERSON

```
INSERT INTO person
(Name, Person_Type, Contact_Info, Date_of_Birth, Identification_No)
VALUES
('Rahul Verma', 'Witness', '9876500001', '1995-05-12', 'ID1001'),
('Suresh Reddy', 'Suspect', '9876500002', '1990-08-20', 'ID1002'),
('Anita Rao', 'Victim', '9876500003', '1988-02-15', 'ID1003'),
('Kiran Das', 'Witness', '9876500004', '1998-11-25', 'ID1004'),
('Vijay Kumar', 'Suspect', '9876500005', '1992-03-10', 'ID1005');
```

```
mysql> INSERT INTO person
    -> (Name, Person_Type, Contact_Info, Date_of_Birth, Identification_No)
    -> VALUES
    -> ('Rahul Verma', 'Witness', '9876500001', '1995-05-12', 'ID1001'),
    -> ('Suresh Reddy', 'Suspect', '9876500002', '1990-08-20', 'ID1002'),
    -> ('Anita Rao', 'Victim', '9876500003', '1988-02-15', 'ID1003'),
    -> ('Kiran Das', 'Witness', '9876500004', '1998-11-25', 'ID1004'),
    -> ('Vijay Kumar', 'Suspect', '9876500005', '1992-03-10', 'ID1005');
Query OK, 5 rows affected (0.03 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM person;
+-----------+--------------+-------------+--------------+---------------+-------------------+
| Person_ID | Name         | Person_Type | Contact_Info | Date_of_Birth | Identification_No |
+-----------+--------------+-------------+--------------+---------------+-------------------+
|         1 | Rahul Verma  | Witness     | 9876500001   | 1995-05-12    | ID1001            |
|         2 | Suresh Reddy | Suspect     | 9876500002   | 1990-08-20    | ID1002            |
|         3 | Anita Rao    | Victim      | 9876500003   | 1988-02-15    | ID1003            |
|         4 | Kiran Das    | Witness     | 9876500004   | 1998-11-25    | ID1004            |
|         5 | Vijay Kumar  | Suspect     | 9876500005   | 1992-03-10    | ID1005            |
+-----------+--------------+-------------+--------------+---------------+-------------------+
5 rows in set (0.00 sec)
```

Figure 6.5: Records inserted into person

6.6 INVOLVES

```
INSERT INTO involves (Case_ID, Person_ID)
VALUES
(1, 1),
(1, 2),
(1, 3),
(2, 4),
(2, 5),
(3, 1);
```

```
mysql> INSERT INTO involves (Case_ID, Person_ID)
    -> VALUES
    -> (1, 1),
    -> (1, 2),
    -> (1, 3),
    -> (2, 4),
    -> (2, 5),
    -> (3, 1);
Query OK, 6 rows affected (0.03 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM involves;
+---------+-----------+
| Case_ID | Person_ID |
+---------+-----------+
|       1 |         1 |
|       3 |         1 |
|       1 |         2 |
|       1 |         3 |
|       2 |         4 |
|       2 |         5 |
+---------+-----------+
6 rows in set (0.00 sec)
```

Figure 6.6: Records inserted into involves

6.7 EVIDENCE

```
INSERT INTO evidence
(Case_ID, Category_ID, Description, Collected_Date,
 Collected_By, Current_Status, Storage_Location,
 Hash_Value, Hash_Algorithm)
VALUES
(1, 1,
 'Mobile phone recovered from crime scene',
 '2026-09-01', 1, 'Stored',
 'Evidence Storage Room', 'HASH001', 'SHA256'),


(1, 2,
 'Metal tool recovered near entrance',
 '2026-09-01', 1, 'In_Lab',
 'Forensic Science Laboratory', 'HASH002', 'SHA256'),


(1, 1,
 'CCTV footage from jewellery store',
 '2026-09-02', 3, 'Stored',
 'Evidence Storage Room', 'HASH003', 'SHA256'),


(2, 2,
 'Vehicle key recovered from suspect',
 '2026-09-05', 3, 'Collected',
 'Central Police Station', 'HASH004', 'SHA256'),


(2, 1,
 'GPS device recovered from vehicle',
 '2026-09-06', 3, 'Stored',
 'Evidence Storage Room', 'HASH005', 'SHA256'),


(3, 4,
 'Bank transaction document',
 '2026-09-10', 1, 'Stored',
 'Evidence Storage Room', 'HASH006', 'SHA256');
```

```
mysql> INSERT INTO evidence
    -> (Case_ID, Category_ID, Description, Collected_Date,
    ->  Collected_By, Current_Status, Storage_Location,
    ->  Hash_Value, Hash_Algorithm)
    -> VALUES
    -> (1, 1,
    ->  'Mobile phone recovered from crime scene',
    ->  '2026-09-01', 1, 'Stored',
    ->  'Evidence Storage Room', 'HASH001', 'SHA256'),
    -> (1, 2,
    ->  'Metal tool recovered near entrance',
    ->  '2026-09-01', 1, 'In_Lab',
    ->  'Forensic Science Laboratory', 'HASH002', 'SHA256'),
    -> (1, 1,
    ->  'CCTV footage from jewellery store',
    ->  '2026-09-02', 3, 'Stored',
    ->  'Evidence Storage Room', 'HASH003', 'SHA256'),
    -> (2, 2,
    ->  'Vehicle key recovered from suspect',
    ->  '2026-09-05', 3, 'Collected',
    ->  'Central Police Station', 'HASH004', 'SHA256'),
    -> (2, 1,
    ->  'GPS device recovered from vehicle',
    ->  '2026-09-06', 3, 'Stored',
    ->  'Evidence Storage Room', 'HASH005', 'SHA256'),
    -> (3, 4,
    ->  'Bank transaction document',
    ->  '2026-09-10', 1, 'Stored',
    ->  'Evidence Storage Room', 'HASH006', 'SHA256');
Query OK, 6 rows affected (0.04 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM evidence;
+-------------+---------+-------------+-----------------------------------+----------------+---------------+-----------------------------+-------------+----------------+-----------------+
| Evidence_ID | Case_ID | Category_ID | Description                       | Collected_Date | Collected_By | Current_Status | Storage_Location            | Hash_Value | Hash_Algorithm | Chain_Verified |
+-------------+---------+-------------+-----------------------------------+----------------+---------------+-----------------------------+-------------+----------------+-----------------+
|           1 |       1 |           1 | Mobile phone recovered from crime scene | 2026-09-01 |             1 | Stored         | Evidence Storage Room       | HASH001    | SHA256         |               1 |
|           2 |       1 |           2 | Metal tool recovered near entrance | 2026-09-01 |             1 | In_Lab         | Forensic Science Laboratory | HASH002    | SHA256         |               1 |
|           3 |       1 |           1 | CCTV footage from jewellery store | 2026-09-02 |             3 | Stored         | Evidence Storage Room       | HASH003    | SHA256         |               1 |
|           4 |       2 |           2 | Vehicle key recovered from suspect | 2026-09-05 |             3 | Collected      | Central Police Station      | HASH004    | SHA256         |               1 |
|           5 |       2 |           1 | GPS device recovered from vehicle | 2026-09-06 |             3 | Stored         | Evidence Storage Room       | HASH005    | SHA256         |               1 |
|           6 |       3 |           4 | Bank transaction document         | 2026-09-10 |             1 | Stored         | Evidence Storage Room       | HASH006    | SHA256         |               1 |
+-------------+---------+-------------+-----------------------------------+----------------+---------------+-----------------------------+-------------+----------------+-----------------+
6 rows in set (0.00 sec)
```

Figure 6.7: Records inserted into evidence

6.8 BELONGS_TO

```
INSERT INTO belongs_to (Person_ID, Evidence_ID)
VALUES
(2, 2),
(3, 1),
(4, 3),
(5, 4);
```

```
mysql> INSERT INTO belongs_to (Person_ID, Evidence_ID)
    -> VALUES
    -> (2, 2),
    -> (3, 1),
    -> (4, 3),
    -> (5, 4);
Query OK, 4 rows affected (0.04 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM belongs_to;
+-----------+-------------+
| Person_ID | Evidence_ID |
+-----------+-------------+
|         3 |           1 |
|         2 |           2 |
|         4 |           3 |
|         5 |           4 |
+-----------+-------------+
4 rows in set (0.00 sec)
```

Figure 6.8: Records inserted into belongs_to

6.9 FORENSIC_TEST

```
INSERT INTO forensic_test
(Evidence_ID, Test_Type, Conducted_By, Test_Date,
 Result, Report_Path)
VALUES
(1, 'Mobile Data Extraction', 2,
 '2026-09-03',
 'Relevant communication records recovered',
 '/reports/test001.pdf'),

(2, 'Fingerprint Analysis', 2,
 '2026-09-03',
 'Partial fingerprint detected',
 '/reports/test002.pdf'),

(3, 'Video Analysis', 2,
 '2026-09-04',
 'Suspected individual identified in footage',
 '/reports/test003.pdf'),

(5, 'GPS Data Analysis', 2,
 '2026-09-07',
 'Vehicle movement history recovered',
 '/reports/test004.pdf');
```

```
mysql> INSERT INTO forensic_test
    -> (Evidence_ID, Test_Type, Conducted_By, Test_Date,
    ->  Result, Report_Path)
    -> VALUES
    -> (1, 'Mobile Data Extraction', 2,
    ->  '2026-09-03',
    ->  'Relevant communication records recovered',
    ->  '/reports/test001.pdf'),
    -> (2, 'Fingerprint Analysis', 2,
    ->  '2026-09-03',
    ->  'Partial fingerprint detected',
    ->  '/reports/test002.pdf'),
    -> (3, 'Video Analysis', 2,
    ->  '2026-09-04',
    ->  'Suspected individual identified in footage',
    ->  '/reports/test003.pdf'),
    -> (5, 'GPS Data Analysis', 2,
    ->  '2026-09-07',
    ->  'Vehicle movement history recovered',
    ->  '/reports/test004.pdf');
Query OK, 4 rows affected (0.02 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM forensic_test;
+---------+-------------+----------------------+-------------+------------+-----------------------------------------+----------------------+
| Test_ID | Evidence_ID | Test_Type            | Conducted_By | Test_Date  | Result                                  | Report_Path          |
+---------+-------------+----------------------+-------------+------------+-----------------------------------------+----------------------+
|       1 |           1 | Mobile Data Extraction |           2 | 2026-09-03 | Relevant communication records recovered   | /reports/test001.pdf |
|       2 |           2 | Fingerprint Analysis   |           2 | 2026-09-03 | Partial fingerprint detected               | /reports/test002.pdf |
|       3 |           3 | Video Analysis         |           2 | 2026-09-04 | Suspected individual identified in footage | /reports/test003.pdf |
|       4 |           5 | GPS Data Analysis      |           2 | 2026-09-07 | Vehicle movement history recovered         | /reports/test004.pdf |
+---------+-------------+----------------------+-------------+------------+-----------------------------------------+----------------------+
4 rows in set (0.00 sec)
```

Figure 6.9: Records inserted into forensic_test

6.10 CUSTODY_TRANSFER

```
INSERT INTO custody_transfer
(Evidence_ID, Transfer_No, From_User_ID, To_User_ID,
 Transfer_DateTime, Received_Time, From_Location_ID, Remarks)
VALUES
(1, 1, 1, 4,
 '2026-09-01 10:00:00',
 '2026-09-01 10:30:00',
 1, 'Evidence submitted to storage'),

(1, 2, 4, 2,
 '2026-09-03 09:00:00',
 '2026-09-03 09:30:00',
 2, 'Sent for forensic examination'),

(1, 3, 2, 4,
 '2026-09-04 16:00:00',
 '2026-09-04 16:30:00',
 3, 'Returned after forensic examination'),

(2, 1, 1, 2,
 '2026-09-02 09:00:00',
 '2026-09-02 09:30:00',
 1, 'Fingerprint examination'),

(3, 1, 3, 4,
 '2026-09-02 11:00:00',
 '2026-09-02 11:30:00',
 1, 'CCTV secured');
```

```
mysql> INSERT INTO custody_transfer
    -> (Evidence_ID, Transfer_No, From_User_ID, To_User_ID,
    ->  Transfer_DateTime, Received_Time, From_Location_ID, Remarks)
    -> VALUES
    -> (1, 1, 1, 4,
    ->  '2026-09-01 10:00:00',
    ->  '2026-09-01 10:30:00',
    ->  1, 'Evidence submitted to storage'),
    -> (1, 2, 4, 2,
    ->  '2026-09-03 09:00:00',
    ->  '2026-09-03 09:30:00',
    ->  2, 'Sent for forensic examination'),
    -> (1, 3, 2, 4,
    ->  '2026-09-04 16:00:00',
    ->  '2026-09-04 16:30:00',
    ->  3, 'Returned after forensic examination'),
    -> (2, 1, 1, 2,
    ->  '2026-09-02 09:00:00',
    ->  '2026-09-02 09:30:00',
    ->  1, 'Fingerprint examination'),
    -> (3, 1, 3, 4,
    ->  '2026-09-02 11:00:00',
    ->  '2026-09-02 11:30:00',
    ->  1, 'CCTV secured');
Query OK, 5 rows affected (0.04 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM custody_transfer;
+-------------+-------------+-------------+--------------+------------+---------------------+---------------------+-----------------+-----------------------------------+
| Transfer_ID | Evidence_ID | Transfer_No | From_User_ID | To_User_ID | Transfer_DateTime   | Received_Time       | From_Location_ID | Remarks                           |
+-------------+-------------+-------------+--------------+------------+---------------------+---------------------+-----------------+-----------------------------------+
|           1 |           1 |           1 |            1 |          4 | 2026-09-01 10:00:00 | 2026-09-01 10:30:00 |               1 | Evidence submitted to storage     |
|           2 |           1 |           2 |            4 |          2 | 2026-09-03 09:00:00 | 2026-09-03 09:30:00 |               2 | Sent for forensic examination     |
|           3 |           1 |           3 |            2 |          4 | 2026-09-04 16:00:00 | 2026-09-04 16:30:00 |               3 | Returned after forensic examination |
|           4 |           2 |           1 |            1 |          2 | 2026-09-02 09:00:00 | 2026-09-02 09:30:00 |               1 | Fingerprint examination           |
|           5 |           3 |           1 |            3 |          4 | 2026-09-02 11:00:00 | 2026-09-02 11:30:00 |               1 | CCTV secured                      |
+-------------+-------------+-------------+--------------+------------+---------------------+---------------------+-----------------+-----------------------------------+
5 rows in set (0.00 sec)
```

Figure 6.10: Records inserted into custody_transfer

7. QUERIES (SIMPLE QUERY AND UPDATE AND DELETE OPERATION, CORRELATED QUERY AND NESTED QUERY)

7.1 Simple queries with GROUP BY and aggregate functions

(a) SELECT with WHERE clause

```
SELECT Evidence_ID, Description, Current_Status
FROM evidence
WHERE Current_Status = 'Stored';
```

```
mysql> SELECT Evidence_ID, Description, Current_Status
    -> FROM evidence
    -> WHERE Current_Status = 'Stored';
+-------------+----------------------------------------+----------------+
| Evidence_ID | Description                            | Current_Status |
+-------------+----------------------------------------+----------------+
|           1 | Mobile phone recovered from crime scene | Stored         |
|           3 | CCTV footage from jewellery store      | Stored         |
|           5 | GPS device recovered from vehicle      | Stored         |
|           6 | Bank transaction document              | Stored         |
+-------------+----------------------------------------+----------------+
4 rows in set (0.00 sec)

mysql>
```

Figure 7.1: (a) SELECT with WHERE clause

(b) ORDER BY

```
SELECT Evidence_ID, Description, Collected_Date
FROM evidence
ORDER BY Collected_Date DESC;
```

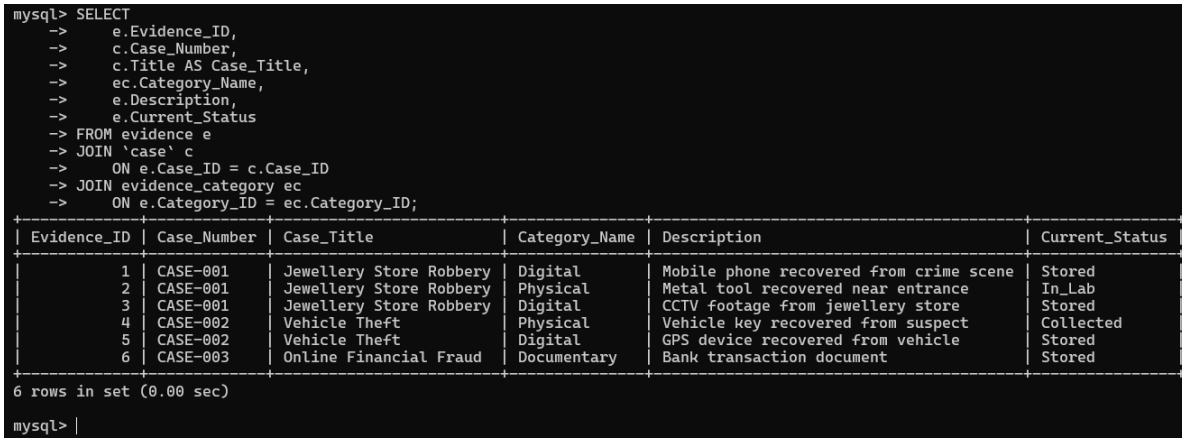
```
mysql> SELECT Evidence_ID, Description, Collected_Date
    -> FROM evidence
    -> ORDER BY Collected_Date DESC;
+-------------+----------------------------------------+----------------+
| Evidence_ID | Description                            | Collected_Date |
+-------------+----------------------------------------+----------------+
|           6 | Bank transaction document              | 2026-09-10     |
|           5 | GPS device recovered from vehicle      | 2026-09-06     |
|           4 | Vehicle key recovered from suspect     | 2026-09-05     |
|           3 | CCTV footage from jewellery store      | 2026-09-02     |
|           1 | Mobile phone recovered from crime scene | 2026-09-01     |
|           2 | Metal tool recovered near entrance     | 2026-09-01     |
+-------------+----------------------------------------+----------------+
6 rows in set (0.00 sec)

mysql>
```

Figure 7.2: (b) ORDER BY

(c) JOIN of evidence, case and category

```
SELECT
    e.Evidence_ID,
    c.Case_Number,
    c.Title AS Case_Title,
    ec.Category_Name,
    e.Description,
    e.Current_Status
FROM evidence e
JOIN `case` c
    ON e.Case_ID = c.Case_ID
JOIN evidence_category ec
    ON e.Category_ID = ec.Category_ID;
```



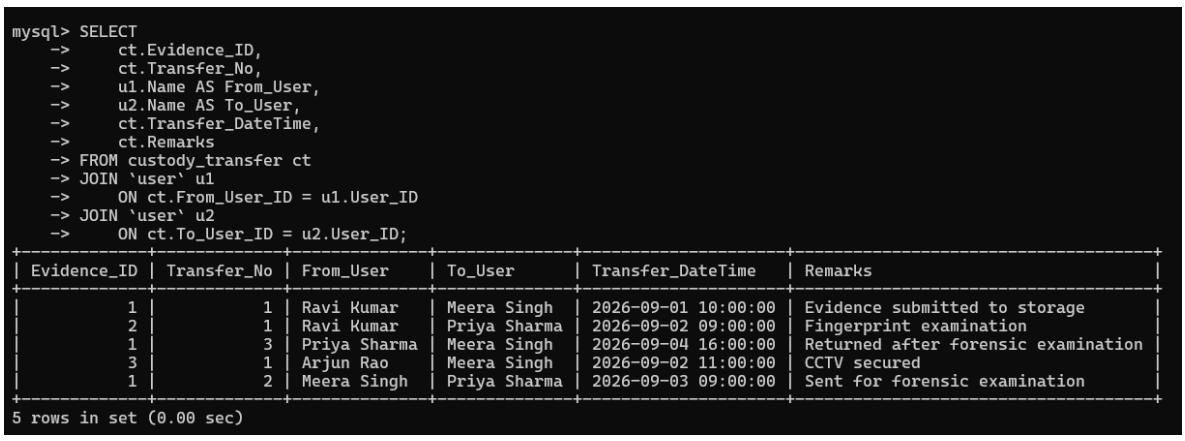
```
mysql> SELECT
    ->     e.Evidence_ID,
    ->     c.Case_Number,
    ->     c.Title AS Case_Title,
    ->     ec.Category_Name,
    ->     e.Description,
    ->     e.Current_Status
    -> FROM evidence e
    -> JOIN `case` c
    ->     ON e.Case_ID = c.Case_ID
    -> JOIN evidence_category ec
    ->     ON e.Category_ID = ec.Category_ID;
+-------------+-------------+----------------------+---------------+-------------------------------------+----------------+
| Evidence_ID | Case_Number | Case_Title           | Category_Name | Description                         | Current_Status |
+-------------+-------------+----------------------+---------------+-------------------------------------+----------------+
|           1 | CASE-001    | Jewellery Store Robbery | Digital       | Mobile phone recovered from crime scene | Stored         |
|           2 | CASE-001    | Jewellery Store Robbery | Physical      | Metal tool recovered near entrance  | In_Lab         |
|           3 | CASE-001    | Jewellery Store Robbery | Digital       | CCTV footage from jewellery store   | Stored         |
|           4 | CASE-002    | Vehicle Theft        | Physical      | Vehicle key recovered from suspect  | Collected      |
|           5 | CASE-002    | Vehicle Theft        | Digital       | GPS device recovered from vehicle   | Stored         |
|           6 | CASE-003    | Online Financial Fraud | Documentary   | Bank transaction document           | Stored         |
+-------------+-------------+----------------------+---------------+-------------------------------------+----------------+
6 rows in set (0.00 sec)

mysql>
```

Figure 7.3: (c) JOIN of evidence, case and category

(d) JOIN of custody transfers with both users

```
SELECT
    ct.Evidence_ID,
    ct.Transfer_No,
    u1.Name AS From_User,
    u2.Name AS To_User,
    ct.Transfer_DateTime,
    ct.Remarks
FROM custody_transfer ct
JOIN `user` u1
    ON ct.From_User_ID = u1.User_ID
JOIN `user` u2
    ON ct.To_User_ID = u2.User_ID;
```



```
mysql> SELECT
    ->     ct.Evidence_ID,
    ->     ct.Transfer_No,
    ->     u1.Name AS From_User,
    ->     u2.Name AS To_User,
    ->     ct.Transfer_DateTime,
    ->     ct.Remarks
    -> FROM custody_transfer ct
    -> JOIN `user` u1
    ->     ON ct.From_User_ID = u1.User_ID
    -> JOIN `user` u2
    ->     ON ct.To_User_ID = u2.User_ID;
+-------------+-------------+-------------+-------------+---------------------+-----------------------------------+
| Evidence_ID | Transfer_No | From_User   | To_User     | Transfer_DateTime   | Remarks                           |
+-------------+-------------+-------------+-------------+---------------------+-----------------------------------+
|           1 |           1 | Ravi Kumar  | Meera Singh | 2026-09-01 10:00:00 | Evidence submitted to storage     |
|           2 |           1 | Ravi Kumar  | Priya Sharma | 2026-09-02 09:00:00 | Fingerprint examination           |
|           1 |           3 | Priya Sharma | Meera Singh | 2026-09-04 16:00:00 | Returned after forensic examination |
|           3 |           1 | Arjun Rao   | Meera Singh | 2026-09-02 11:00:00 | CCTV secured                      |
|           1 |           2 | Meera Singh | Priya Sharma | 2026-09-03 09:00:00 | Sent for forensic examination     |
+-------------+-------------+-------------+-------------+---------------------+-----------------------------------+
5 rows in set (0.00 sec)
```

Figure 7.4: (d) JOIN of custody transfers with both users

(e) GROUP BY with COUNT

```
SELECT
    ec.Category_Name,
    COUNT(e.Evidence_ID) AS Total_Evidence
FROM evidence e
JOIN evidence_category ec
    ON e.Category_ID = ec.Category_ID
GROUP BY ec.Category_Name;
```

```
mysql> SELECT
    ->     ec.Category_Name,
    ->     COUNT(e.Evidence_ID) AS Total_Evidence
    -> FROM evidence e
    -> JOIN evidence_category ec
    ->     ON e.Category_ID = ec.Category_ID
    -> GROUP BY ec.Category_Name;
+---------------+----------------+
| Category_Name | Total_Evidence |
+---------------+----------------+
| Digital       |              3 |
| Documentary   |              1 |
| Physical      |              2 |
+---------------+----------------+
3 rows in set (0.00 sec)

mysql>
```

Figure 7.5: (e) GROUP BY with COUNT

(f) GROUP BY with HAVING

```
SELECT
    ec.Category_Name,
    COUNT(e.Evidence_ID) AS Total_Evidence
FROM evidence e
JOIN evidence_category ec
    ON e.Category_ID = ec.Category_ID
GROUP BY ec.Category_Name
HAVING COUNT(e.Evidence_ID) > 1;
```

```
mysql> SELECT
    ->     ec.Category_Name,
    ->     COUNT(e.Evidence_ID) AS Total_Evidence
    -> FROM evidence e
    -> JOIN evidence_category ec
    ->     ON e.Category_ID = ec.Category_ID
    -> GROUP BY ec.Category_Name
    -> HAVING COUNT(e.Evidence_ID) > 1;
+---------------+----------------+
| Category_Name | Total_Evidence |
+---------------+----------------+
| Digital       |              3 |
| Physical      |              2 |
+---------------+----------------+
2 rows in set (0.00 sec)
```

Figure 7.6: (f) GROUP BY with HAVING

(g) Evidence count per case (LEFT JOIN)

```
SELECT
    c.Case_Number,
    c.Title,
    COUNT(e.Evidence_ID) AS Total_Evidence
FROM `case` c
LEFT JOIN evidence e
    ON c.Case_ID = e.Case_ID
GROUP BY c.Case_ID, c.Case_Number, c.Title;
```

```
mysql> SELECT
    ->     c.Case_Number,
    ->     c.Title,
    ->     COUNT(e.Evidence_ID) AS Total_Evidence
    -> FROM `case` c
    -> LEFT JOIN evidence e
    ->     ON c.Case_ID = e.Case_ID
    -> GROUP BY c.Case_ID, c.Case_Number, c.Title;
+-------------+-------------------------+----------------+
| Case_Number | Title                   | Total_Evidence |
+-------------+-------------------------+----------------+
| CASE-001    | Jewellery Store Robbery |              3 |
| CASE-002    | Vehicle Theft           |              2 |
| CASE-003    | Online Financial Fraud  |              1 |
+-------------+-------------------------+----------------+
3 rows in set (0.01 sec)
```

Figure 7.7: (g) Evidence count per case (LEFT JOIN)

(h) COUNT, MIN and MAX together

```
SELECT
    COUNT(*) AS Total_Evidence,
    MIN(Collected_Date) AS First_Collection,
    MAX(Collected_Date) AS Latest_Collection
FROM evidence;
```

```
mysql> SELECT
    ->     COUNT(*) AS Total_Evidence,
    ->     MIN(Collected_Date) AS First_Collection,
    ->     MAX(Collected_Date) AS Latest_Collection
    -> FROM evidence;
+----------------+------------------+-------------------+
| Total_Evidence | First_Collection | Latest_Collection |
+----------------+------------------+-------------------+
|              6 | 2026-09-01       | 2026-09-10        |
+----------------+------------------+-------------------+
1 row in set (0.00 sec)
```

Figure 7.8: (h) COUNT, MIN and MAX together

7.2 Update operation

(a) Update evidence status

```
UPDATE evidence
SET Current_Status = 'In_Lab'
WHERE Evidence_ID = 1;

SELECT Evidence_ID, Description, Current_Status
FROM evidence
WHERE Evidence_ID = 1;

-- Restore the sample value
UPDATE evidence
SET Current_Status = 'Stored'
WHERE Evidence_ID = 1;
```

```
mysql> UPDATE evidence
    -> SET Current_Status = 'In_Lab'
    -> WHERE Evidence_ID = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT Evidence_ID, Description, Current_Status
    -> FROM evidence
    -> WHERE Evidence_ID = 1;
+-------------+-----------------------------------------+----------------+
| Evidence_ID | Description                             | Current_Status |
+-------------+-----------------------------------------+----------------+
|           1 | Mobile phone recovered from crime scene | In_Lab         |
+-------------+-----------------------------------------+----------------+
1 row in set (0.00 sec)

mysql> -- Restore the sample value
mysql> UPDATE evidence
    -> SET Current_Status = 'Stored'
    -> WHERE Evidence_ID = 1;
Query OK, 1 row affected (0.03 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Figure 7.9: (a) Update evidence status

(b) Update case status

```
UPDATE `case`
SET Status = 'Under Investigation'
WHERE Case_ID = 1;

SELECT Case_ID, Case_Number, Status
FROM `case`
WHERE Case_ID = 1;
```

```
mysql> SELECT Case_ID, Case_Number, Status
    -> FROM `case`
    -> WHERE Case_ID = 1;
+---------+-------------+--------------------+
| Case_ID | Case_Number | Status             |
+---------+-------------+--------------------+
|       1 | CASE-001    | Under Investigation |
+---------+-------------+--------------------+
1 row in set (0.00 sec)

mysql>
```

Figure 7.10: (b) Update case status

7.3 Delete operation

Delete a forensic test record

```
DELETE FROM forensic_test
WHERE Test_ID = 4;


SELECT *
FROM forensic_test;
```

```
mysql> DELETE FROM forensic_test
    -> WHERE Test_ID = 4;
Query OK, 1 row affected (0.01 sec)

mysql> SELECT *
    -> FROM forensic_test;
+---------+------------+----------------------+--------------+------------+-----------------------------------------+----------------------+
| Test_ID | Evidence_ID | Test_Type            | Conducted_By | Test_Date  | Result                                  | Report_Path          |
+---------+------------+----------------------+--------------+------------+-----------------------------------------+----------------------+
|       1 |          1 | Mobile Data Extraction |            2 | 2026-09-03 | Relevant communication records recovered | /reports/test001.pdf |
|       2 |          2 | Fingerprint Analysis   |            2 | 2026-09-03 | Partial fingerprint detected            | /reports/test002.pdf |
|       3 |          3 | Video Analysis         |            2 | 2026-09-04 | Suspected individual identified in footage | /reports/test003.pdf |
+---------+------------+----------------------+--------------+------------+-----------------------------------------+----------------------+
3 rows in set (0.00 sec)

mysql>
```

Figure 7.11: Delete a forensic test record

7.4 Correlated query

A correlated subquery refers to a column of the outer query, so it is evaluated once for every outer row.

(a) Latest evidence collected in each case

```
SELECT e.Evidence_ID, e.Description, e.Case_ID, e.Collected_Date
FROM evidence e
WHERE e.Collected_Date =
(
    SELECT MAX(e2.Collected_Date)
    FROM evidence e2
    WHERE e2.Case_ID = e.Case_ID
);
```

```
mysql> SELECT e.Evidence_ID, e.Description, e.Case_ID, e.Collected_Date
    -> FROM evidence e
    -> WHERE e.Collected_Date =
    -> (
    ->     SELECT MAX(e2.Collected_Date)
    ->     FROM evidence e2
    ->     WHERE e2.Case_ID = e.Case_ID
    -> );
+-------------+-----------------------------------+---------+----------------+
| Evidence_ID | Description                       | Case_ID | Collected_Date |
+-------------+-----------------------------------+---------+----------------+
|           3 | CCTV footage from jewellery store |       1 | 2026-09-02     |
|           5 | GPS device recovered from vehicle |       2 | 2026-09-06     |
|           6 | Bank transaction document         |       3 | 2026-09-10     |
+-------------+-----------------------------------+---------+----------------+
3 rows in set (0.01 sec)
```

Figure 7.12: (a) Latest evidence collected in each case

(b) Number of custody transfers for each evidence item

```
SELECT e.Evidence_ID, e.Description,
    (
        SELECT COUNT(*)
        FROM custody_transfer ct
        WHERE ct.Evidence_ID = e.Evidence_ID
    ) AS Transfer_Count
FROM evidence e;
```

```
mysql> SELECT e.Evidence_ID, e.Description,
    ->     (
    ->         SELECT COUNT(*)
    ->         FROM custody_transfer ct
    ->         WHERE ct.Evidence_ID = e.Evidence_ID
    ->     ) AS Transfer_Count
    -> FROM evidence e;
+-------------+-----------------------------------------+----------------+
| Evidence_ID | Description                             | Transfer_Count |
+-------------+-----------------------------------------+----------------+
|           1 | Mobile phone recovered from crime scene |              3 |
|           2 | Metal tool recovered near entrance      |              1 |
|           3 | CCTV footage from jewellery store       |              1 |
|           4 | Vehicle key recovered from suspect      |              0 |
|           5 | GPS device recovered from vehicle       |              0 |
|           6 | Bank transaction document               |              0 |
+-------------+-----------------------------------------+----------------+
6 rows in set (0.00 sec)
```

Figure 7.13: (b) Number of custody transfers for each evidence item

7.5 Nested query

(a) Evidence belonging to open cases

```
SELECT Evidence_ID, Description
FROM evidence
WHERE Case_ID IN
(
    SELECT Case_ID
    FROM `case`
    WHERE Status = 'Open'
);
```

```
mysql> SELECT Evidence_ID, Description
    -> FROM evidence
    -> WHERE Case_ID IN
    -> (
    ->     SELECT Case_ID
    ->     FROM `case`
    ->     WHERE Status = 'Open'
    -> );
+-------------+-----------------------------------+
| Evidence_ID | Description                       |
+-------------+-----------------------------------+
|           4 | Vehicle key recovered from suspect |
|           5 | GPS device recovered from vehicle  |
+-------------+-----------------------------------+
2 rows in set (0.00 sec)
```

Figure 7.14: (a) Evidence belonging to open cases

(b) Users who have collected evidence

```
SELECT Name
FROM `user`
WHERE User_ID IN
(
    SELECT DISTINCT Collected_By
    FROM evidence
);
```

```
mysql> SELECT Name
    -> FROM `user`
    -> WHERE User_ID IN
    -> (
    ->     SELECT DISTINCT Collected_By
    ->     FROM evidence
    -> );
+------------+
| Name       |
+------------+
| Ravi Kumar |
| Arjun Rao  |
+------------+
2 rows in set (0.01 sec)
```

Figure 7.15: (b) Users who have collected evidence

(c) Evidence collected after the earliest collection date

```
SELECT Evidence_ID, Description, Collected_Date
FROM evidence
WHERE Collected_Date >
(
    SELECT MIN(Collected_Date)
    FROM evidence
);
```

```
mysql> SELECT Evidence_ID, Description, Collected_Date
    -> FROM evidence
    -> WHERE Collected_Date >
    -> (
    ->     SELECT MIN(Collected_Date)
    ->     FROM evidence
    -> );
+-------------+-----------------------------------+----------------+
| Evidence_ID | Description                       | Collected_Date |
+-------------+-----------------------------------+----------------+
|           3 | CCTV footage from jewellery store | 2026-09-02     |
|           4 | Vehicle key recovered from suspect | 2026-09-05     |
|           5 | GPS device recovered from vehicle | 2026-09-06     |
|           6 | Bank transaction document         | 2026-09-10     |
+-------------+-----------------------------------+----------------+
4 rows in set (0.00 sec)
```

Figure 7.16: (c) Evidence collected after the earliest collection date

8. STORED PROCEDURES, FUNCTIONS AND TRIGGERS

8.1 Stored function – verify custody chain

verify_chain_integrity(p_evidence_id) returns VALID or INVALID for an evidence item. A chain is invalid if any transfer was received before it was handed over, or if a transfer numbered greater than 1 has no preceding transfer number (a gap in the chain). The web application calls this function when the user clicks **Verify** on an evidence record, and sets Chain_Verified to TRUE only when the result is VALID.

```
DROP FUNCTION IF EXISTS verify_chain_integrity;

DELIMITER //

CREATE FUNCTION verify_chain_integrity(p_evidence_id INT)
RETURNS VARCHAR(20)
NOT DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE invalid_count INT DEFAULT 0;

    SELECT COUNT(*)
    INTO invalid_count
    FROM custody_transfer ct
    WHERE ct.Evidence_ID = p_evidence_id
      AND
      (
          ct.Received_Time < ct.Transfer_DateTime

          OR

          (
              ct.Transfer_No > 1
              AND NOT EXISTS
              (
                  SELECT 1
                  FROM custody_transfer previous_transfer
                  WHERE previous_transfer.Evidence_ID = ct.Evidence_ID
                    AND previous_transfer.Transfer_No =
                        ct.Transfer_No - 1
              )
          )
      );

    IF invalid_count = 0 THEN
        RETURN 'VALID';
    ELSE
        RETURN 'INVALID';
    END IF;
END//

DELIMITER ;
```

```
mysql> DELIMITER //
mysql> CREATE FUNCTION verify_chain_integrity(p_evidence_id INT)
    -> RETURNS VARCHAR(20)
    -> NOT DETERMINISTIC
    -> READS SQL DATA
    -> BEGIN
    ->     DECLARE invalid_count INT DEFAULT 0;
    ->     SELECT COUNT(*)
    ->     INTO invalid_count
    ->     FROM custody_transfer ct
    ->     WHERE ct.Evidence_ID = p_evidence_id
    ->       AND
    ->       (
    ->           ct.Received_Time < ct.Transfer_DateTime
    ->           OR
    ->           (
    ->               ct.Transfer_No > 1
    ->               AND NOT EXISTS
    ->               (
    ->                   SELECT 1
    ->                   FROM custody_transfer previous_transfer
    ->                   WHERE previous_transfer.Evidence_ID = ct.Evidence_ID
    ->                     AND previous_transfer.Transfer_No =
    ->                         ct.Transfer_No - 1
    ->               )
    ->           )
    ->       );
    ->     IF invalid_count = 0 THEN
    ->         RETURN 'VALID';
    ->     ELSE
    ->         RETURN 'INVALID';
    ->     END IF;
    -> END//
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
mysql>
```

Figure 8.1: Creation of stored function verify_chain_integrity

Testing the function

```
SELECT verify_chain_integrity(1) AS Chain_Status;
SELECT verify_chain_integrity(2) AS Chain_Status;
```

```
mysql> SELECT verify_chain_integrity(1) AS Chain_Status;
+--------------+
| Chain_Status |
+--------------+
| VALID        |
+--------------+
1 row in set (0.01 sec)

mysql> SELECT verify_chain_integrity(2) AS Chain_Status;
+--------------+
| Chain_Status |
+--------------+
| VALID        |
+--------------+
1 row in set (0.00 sec)
```

Figure 8.2: Testing the function

8.2 Trigger – automatic evidence status update

The AFTER INSERT trigger **trg_after_custody_transfer** fires whenever a custody transfer is logged. It sets the evidence status to In_Transit and clears Chain_Verified, so that the chain must be verified again after every handover.

```
DROP TRIGGER IF EXISTS trg_after_custody_transfer;

DELIMITER //

CREATE TRIGGER trg_after_custody_transfer
AFTER INSERT ON custody_transfer
FOR EACH ROW
BEGIN
    UPDATE evidence
    SET Current_Status = 'In_Transit',
        Chain_Verified = FALSE
    WHERE Evidence_ID = NEW.Evidence_ID;
END//

DELIMITER ;
```

```
mysql> DELIMITER //
mysql> CREATE TRIGGER trg_after_custody_transfer
    -> AFTER INSERT ON custody_transfer
    -> FOR EACH ROW
    -> BEGIN
    ->     UPDATE evidence
    ->     SET Current_Status = 'In_Transit',
    ->         Chain_Verified = FALSE
    ->     WHERE Evidence_ID = NEW.Evidence_ID;
    -> END//
Query OK, 0 rows affected (0.03 sec)

mysql> DELIMITER ;
mysql> SHOW TRIGGERS\G
*************************** 1. row ***************************
             Trigger: trg_after_custody_transfer
               Event: INSERT
               Table: custody_transfer
           Statement: BEGIN
    UPDATE evidence
    SET Current_Status = 'In_Transit',
        Chain_Verified = FALSE
    WHERE Evidence_ID = NEW.Evidence_ID;
END
              Timing: AFTER
             Created: 2026-11-02 18:35:47.85
            sql_mode: ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION
             Definer: root@localhost
character_set_client: cp850
collation_connection: cp850_general_ci
  Database Collation: utf8mb4_0900_ai_ci
1 row in set (0.03 sec)
```

Figure 8.3: Creation of trigger trg_after_custody_transfer

Testing the trigger

```
INSERT INTO custody_transfer
(Evidence_ID, Transfer_No, From_User_ID, To_User_ID,
 Transfer_DateTime, Received_Time, From_Location_ID, Remarks)
VALUES
(3, 2, 1, 2,
 '2026-09-28 11:00:00', '2026-09-28 11:30:00',
 1, 'CCTV footage sent for forensic analysis');

SELECT Evidence_ID, Current_Status, Chain_Verified
FROM evidence
WHERE Evidence_ID = 3;
```

```
mysql> INSERT INTO custody_transfer
    -> (Evidence_ID, Transfer_No, From_User_ID, To_User_ID,
    ->  Transfer_DateTime, Received_Time, From_Location_ID, Remarks)
    -> VALUES
    -> (3, 2, 1, 2,
    ->  '2026-09-28 11:00:00', '2026-09-28 11:30:00',
    ->  1, 'CCTV footage sent for forensic analysis');
Query OK, 1 row affected (0.03 sec)

mysql> SELECT Evidence_ID, Current_Status, Chain_Verified
    -> FROM evidence
    -> WHERE Evidence_ID = 3;
+-------------+----------------+----------------+
| Evidence_ID | Current_Status | Chain_Verified |
+-------------+----------------+----------------+
|           3 | In_Transit     |              0 |
+-------------+----------------+----------------+
1 row in set (0.00 sec)
```

Figure 8.4: Testing the trigger

8.3 Transaction – evidence custody transfer

A transfer and the matching status change must succeed or fail together, so they are executed as one transaction. The web application also allocates the next Transfer_No inside a transaction using SELECT … FOR UPDATE, so two officers cannot create the same transfer number.

Transaction example

```
START TRANSACTION;


INSERT INTO custody_transfer
(Evidence_ID, Transfer_No, From_User_ID, To_User_ID,
 Transfer_DateTime, Received_Time, From_Location_ID, Remarks)
VALUES
(2, 2, 1, 2,
 '2026-09-28 10:00:00', '2026-09-28 10:30:00',
 1, 'Evidence transferred to forensic officer');


UPDATE evidence
SET Current_Status = 'In_Lab'
WHERE Evidence_ID = 2;


COMMIT;


SELECT * FROM custody_transfer WHERE Evidence_ID = 2;
SELECT Evidence_ID, Current_Status FROM evidence WHERE Evidence_ID = 2;
```



Figure 8.5: Transaction example

8.4 View – evidence investigation summary

The view **evidence_summary** joins evidence, case and category so that reports can be produced without repeating the joins.

```
CREATE VIEW evidence_summary AS
SELECT
    e.Evidence_ID,
    c.Case_Number,
    c.Title AS Case_Title,
    ec.Category_Name,
    e.Description,
    e.Current_Status,
    e.Chain_Verified
FROM evidence e
JOIN `case` c
    ON e.Case_ID = c.Case_ID
JOIN evidence_category ec
    ON e.Category_ID = ec.Category_ID;
```

```
mysql> CREATE VIEW evidence_summary AS
    -> SELECT
    ->     e.Evidence_ID,
    ->     c.Case_Number,
    ->     c.Title AS Case_Title,
    ->     ec.Category_Name,
    ->     e.Description,
    ->     e.Current_Status,
    ->     e.Chain_Verified
    -> FROM evidence e
    -> JOIN `case` c
    ->     ON e.Case_ID = c.Case_ID
    -> JOIN evidence_category ec
    ->     ON e.Category_ID = ec.Category_ID;
Query OK, 0 rows affected (0.03 sec)

mysql> SHOW FULL TABLES WHERE Table_type = 'VIEW';
+----------------------------------------+------------+
| Tables_in_legal_evidence_custody_db | Table_type |
+----------------------------------------+------------+
| evidence_summary                       | VIEW       |
+----------------------------------------+------------+
1 row in set (0.01 sec)

mysql> SELECT * FROM evidence_summary;
+-------------+-------------+--------------------------+---------------+-----------------------------------------+----------------+-----------------+
| Evidence_ID | Case_Number | Case_Title               | Category_Name | Description                             | Current_Status | Chain_Verified |
+-------------+-------------+--------------------------+---------------+-----------------------------------------+----------------+-----------------+
|           1 | CASE-001    | Jewellery Store Robbery | Digital       | Mobile phone recovered from crime scene | Stored         |               1 |
|           2 | CASE-001    | Jewellery Store Robbery | Physical      | Metal tool recovered near entrance      | In_Lab         |               0 |
|           3 | CASE-001    | Jewellery Store Robbery | Digital       | CCTV footage from jewellery store       | In_Transit     |               0 |
|           4 | CASE-002    | Vehicle Theft            | Physical      | Vehicle key recovered from suspect      | Collected      |               1 |
|           5 | CASE-002    | Vehicle Theft            | Digital       | GPS device recovered from vehicle       | Stored         |               1 |
|           6 | CASE-003    | Online Financial Fraud   | Documentary   | Bank transaction document               | Stored         |               1 |
+-------------+-------------+--------------------------+---------------+-----------------------------------------+----------------+-----------------+
6 rows in set (0.01 sec)
```

Figure 8.6: Creation and use of view evidence_summary

8.5 Indexing

Indexes speed up the filters used most often by the application: evidence status, case status and transfer date.

```
CREATE INDEX idx_case_status
ON `case`(Status);


CREATE INDEX idx_evidence_status
ON evidence(Current_Status);


CREATE INDEX idx_transfer_datetime
ON custody_transfer(Transfer_DateTime);
```

```
mysql>
mysql> CREATE INDEX idx_case_status
    -> ON `case`(Status);
Query OK, 0 rows affected (0.15 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE INDEX idx_evidence_status
    -> ON evidence(Current_Status);
Query OK, 0 rows affected (0.23 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE INDEX idx_transfer_datetime
    -> ON custody_transfer(Transfer_DateTime);
Query OK, 0 rows affected (0.20 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> SHOW INDEX FROM evidence;
+----------+------------+---------------------------+--------------+---------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table    | Non_unique | Key_name                  | Seq_in_index | Column_name   | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+----------+------------+---------------------------+--------------+---------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| evidence |          0 | PRIMARY                   |            1 | Evidence_ID   | A         |           6 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| evidence |          1 | Case_ID                   |            1 | Case_ID       | A         |           3 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| evidence |          1 | Category_ID               |            1 | Category_ID   | A         |           3 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| evidence |          1 | Collected_By              |            1 | Collected_By  | A         |           2 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| evidence |          1 | idx_evidence_collected_date |          1 | Collected_Date | A        |           5 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| evidence |          1 | idx_evidence_status       |            1 | Current_Status | A        |           4 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+----------+------------+---------------------------+--------------+---------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
6 rows in set (0.03 sec)

mysql> SHOW INDEX FROM custody_transfer;
+-----------------+------------+-------------------------+--------------+-----------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table           | Non_unique | Key_name                | Seq_in_index | Column_name     | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+-----------------+------------+-------------------------+--------------+-----------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| custody_transfer |          0 | PRIMARY                 |            1 | Transfer_ID     | A         |           7 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| custody_transfer |          0 | uq_evidence_transfer    |            1 | Evidence_ID     | A         |           3 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| custody_transfer |          0 | uq_evidence_transfer    |            2 | Transfer_No     | A         |           7 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| custody_transfer |          1 | From_User_ID            |            1 | From_User_ID    | A         |           4 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| custody_transfer |          1 | To_User_ID              |            1 | To_User_ID      | A         |           2 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| custody_transfer |          1 | From_Location_ID        |            1 | From_Location_ID | A        |           3 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| custody_transfer |          1 | idx_transfer_datetime   |            1 | Transfer_DateTime | A       |           7 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+-----------------+------------+-------------------------+--------------+-----------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
7 rows in set (0.02 sec)

mysql> SHOW INDEX FROM `case`;
+-------+------------+----------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table | Non_unique | Key_name       | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+-------+------------+----------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| case  |          0 | PRIMARY        |            1 | Case_ID     | A         |           3 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| case  |          0 | Case_Number    |            1 | Case_Number | A         |           3 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| case  |          1 | idx_case_status |           1 | Status      | A         |           2 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+-------+------------+----------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
3 rows in set (0.05 sec)

mysql> |
```

Figure 8.7: Indexes on evidence, custody_transfer and case

9. FRONT END DEVELOPMENT

9.1 System architecture

The browser talks only to the Next.js server. Page components run in the browser and call REST-style route handlers under /api. Each handler checks the session cookie, validates the request with Zod, runs parameterised SQL against MySQL, and writes an audit event to MongoDB for important actions.

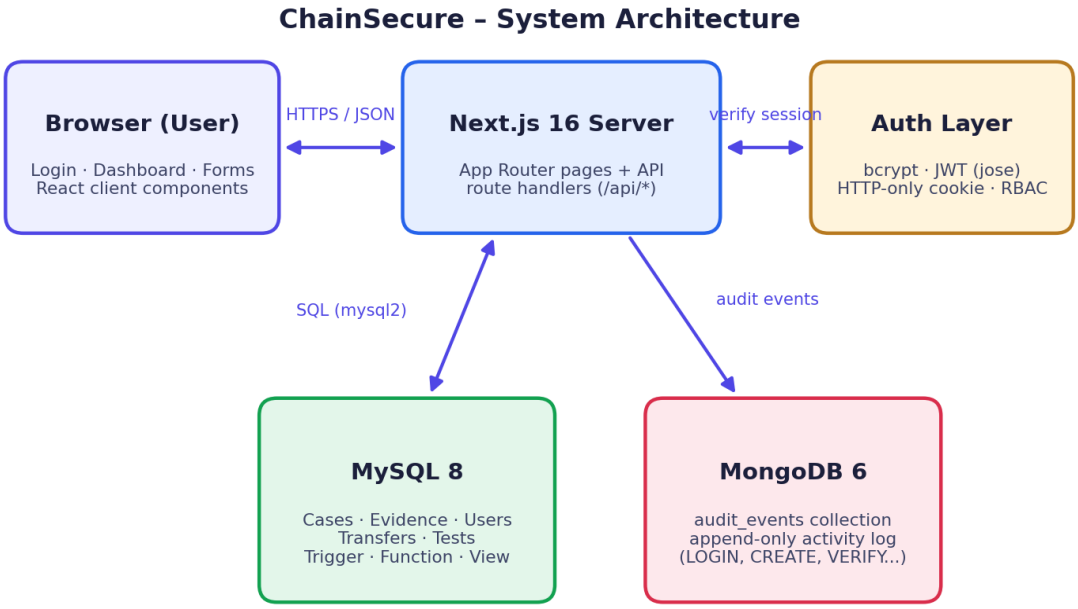


Figure 9.1: System architecture of ChainSecure

9.2 Authentication and security

- Passwords are hashed with bcrypt; plaintext passwords are never stored.
- On successful login the server issues a signed JWT (8-hour expiry) in an HTTP-only, SameSite cookie, so client-side scripts cannot read it.
- Logout calls POST /api/auth/logout, which clears the cookie and writes a LOGOUT event to the audit log.
- Every API route verifies the session and the role on the server; hiding a button in the interface is never the only protection.
- All SQL uses parameterised queries; request bodies are validated with Zod.
- Database credentials, the MongoDB URI and the JWT secret are read from environment variables (.env), which is excluded from version control.

9.3 API endpoints

Table 9.1: REST endpoints of the application

Method and path	Purpose	Access
POST /api/auth/login	Validate credentials and start session	Public
POST /api/auth/logout	End session and log the event	Logged-in
GET /api/evidence	List evidence with case, category and collector	Logged-in
POST /api/evidence	Register evidence	Admin, Officer
DELETE /api/evidence/:id	Delete evidence	Admin, Officer
POST /api/evidence/:id/verify	Run verify_chain_integrity and mark chain verified	Admin, Officer
GET /api/cases	List cases with evidence count	Logged-in
GET, POST /api/transfers	List / log custody transfers (transaction)	View: all; log: Admin, Officer
GET, POST /api/forensic-tests	List / add forensic tests	View: all; add: Admin, Forensic Officer
GET /api/lookups	Dropdown data for forms	Logged-in
GET /api/audit-events	Read the MongoDB audit log	Admin
GET /api/users	List users	Admin

9.4 Application screens

The interface uses a sidebar for navigation, a top bar showing the signed-in user and a Log out button, summary cards, colour-coded status badges, search boxes and modal forms with dropdowns. Capture each screen below from the running application (http://localhost:3000).

9.4.1 Login

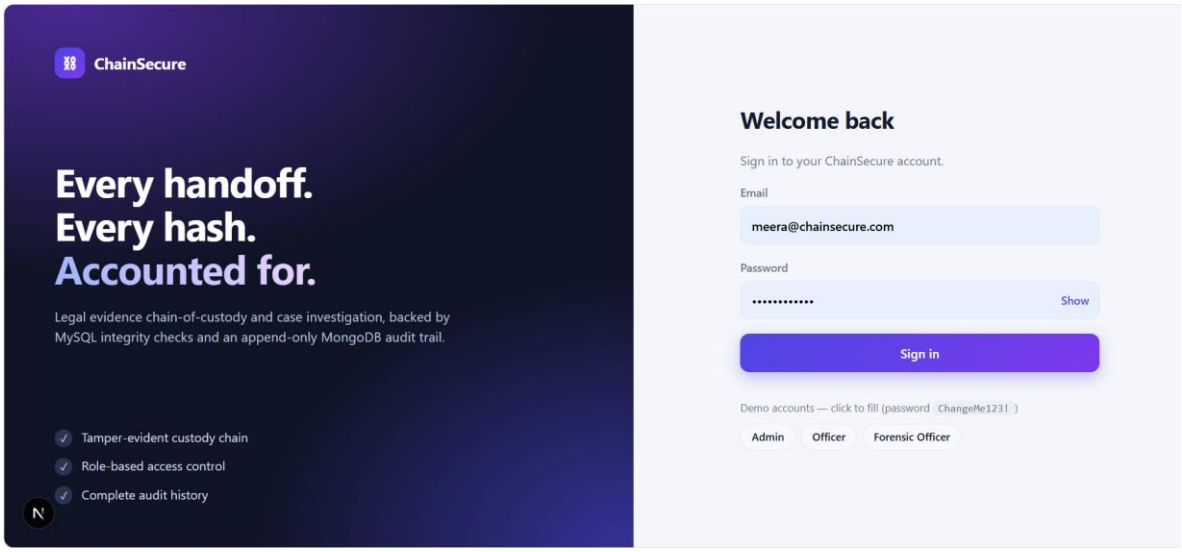


Figure 9.2: Login page

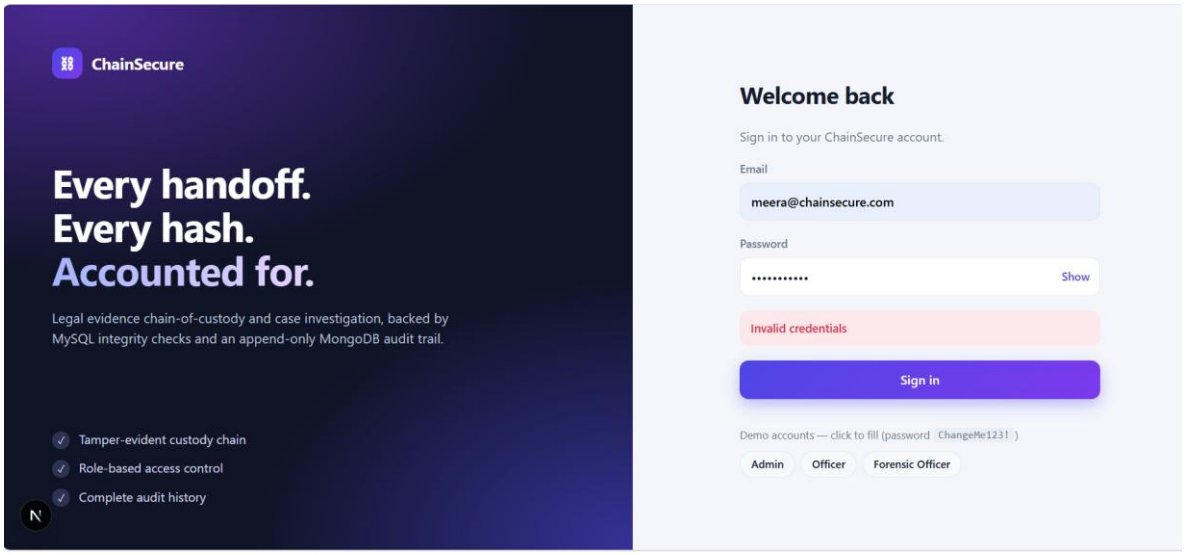


Figure 9.3: Login with invalid credentials

9.4.2 Overview dashboard

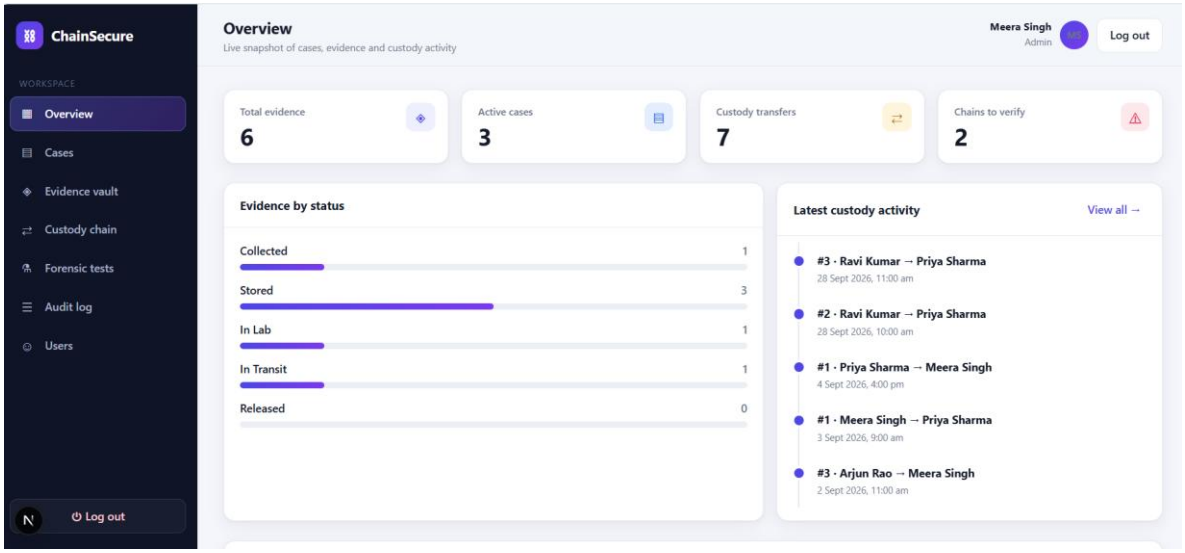


Figure 9.4: Overview dashboard (Admin)

9.4.3 Cases

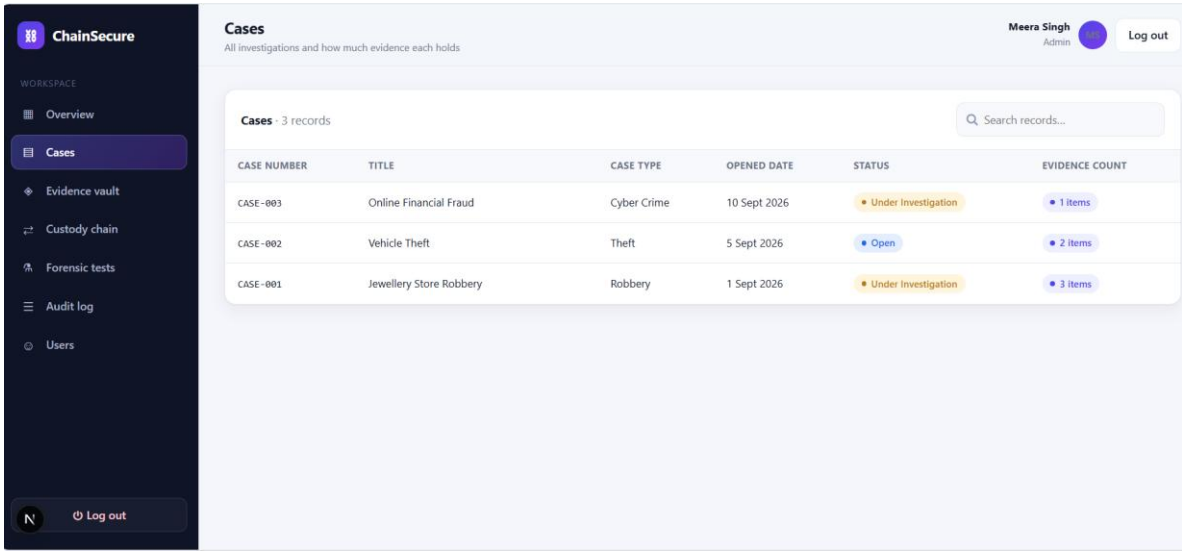


Figure 9.5: Cases page

9.4.4 Evidence vault

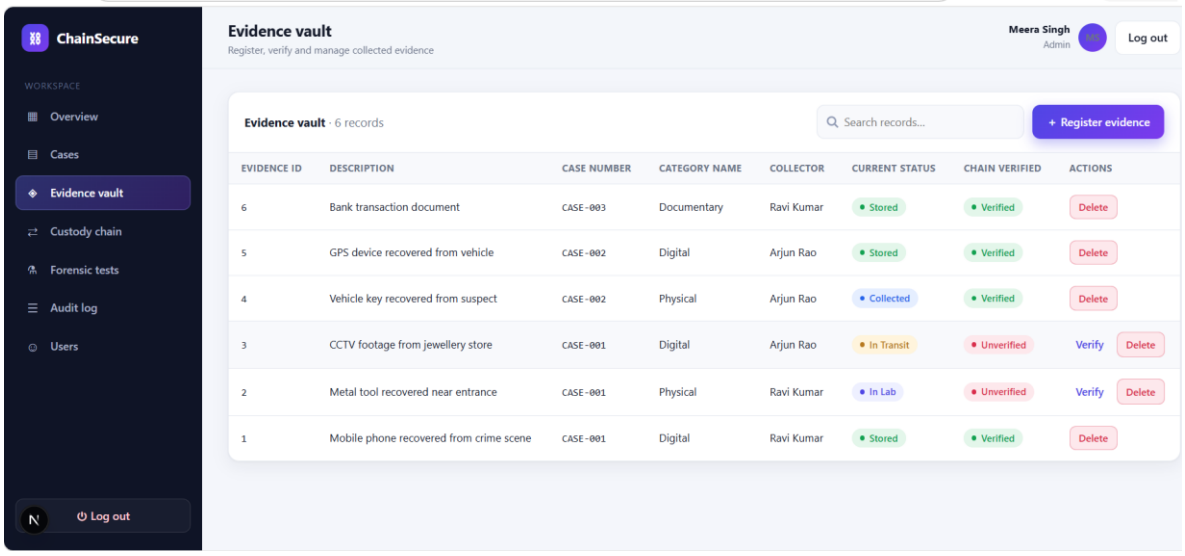


Figure 9.6: Evidence vault

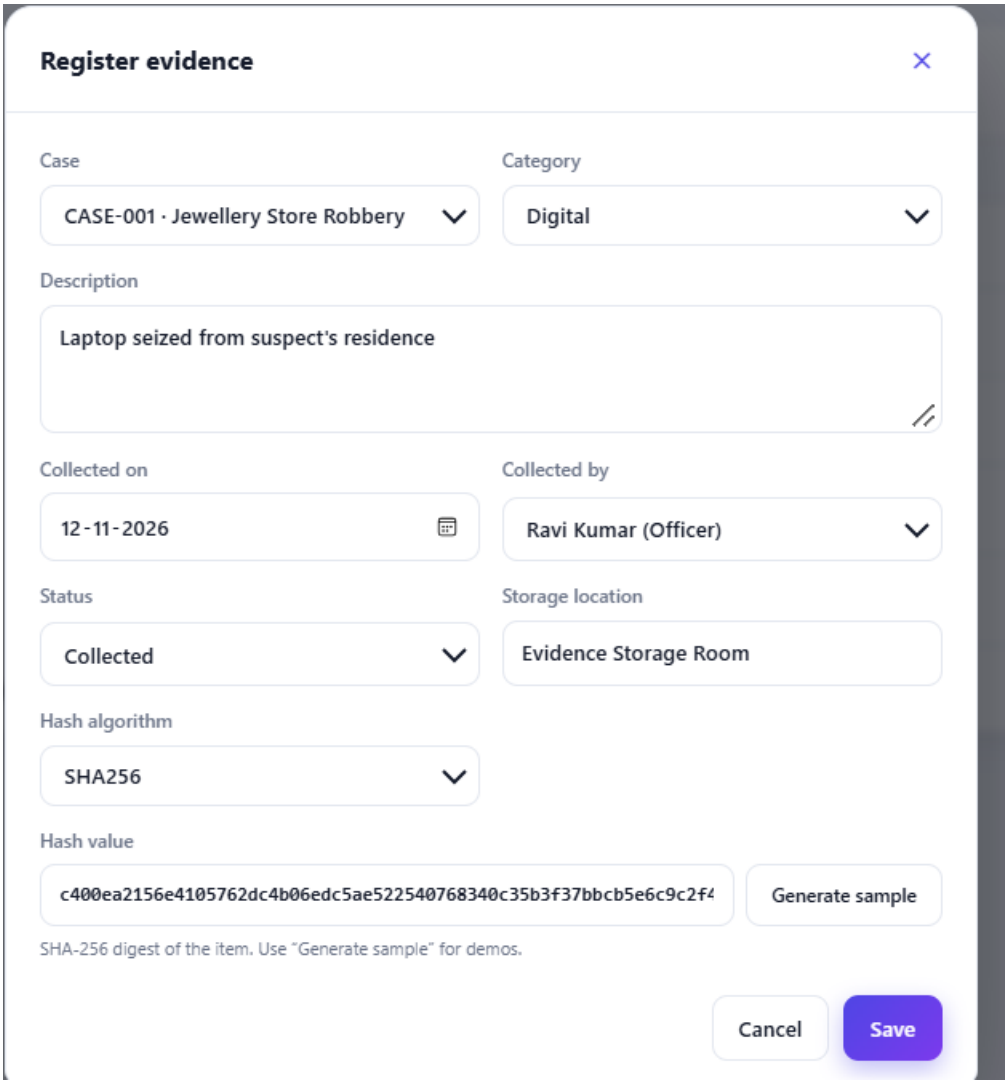


Figure 9.7: Register evidence form

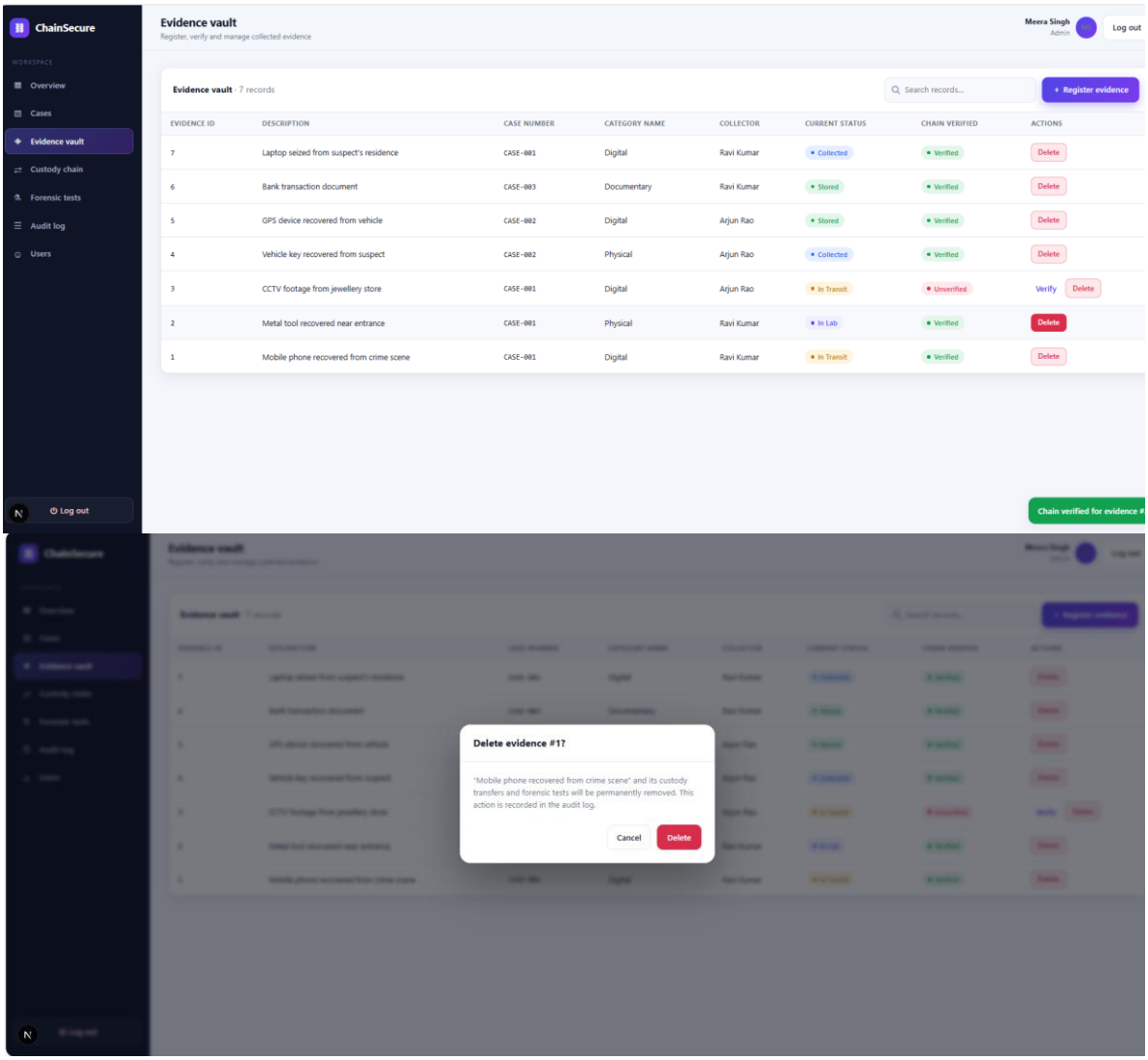


Figure 9.8: Delete confirmation and chain verification

9.4.5 Custody chain

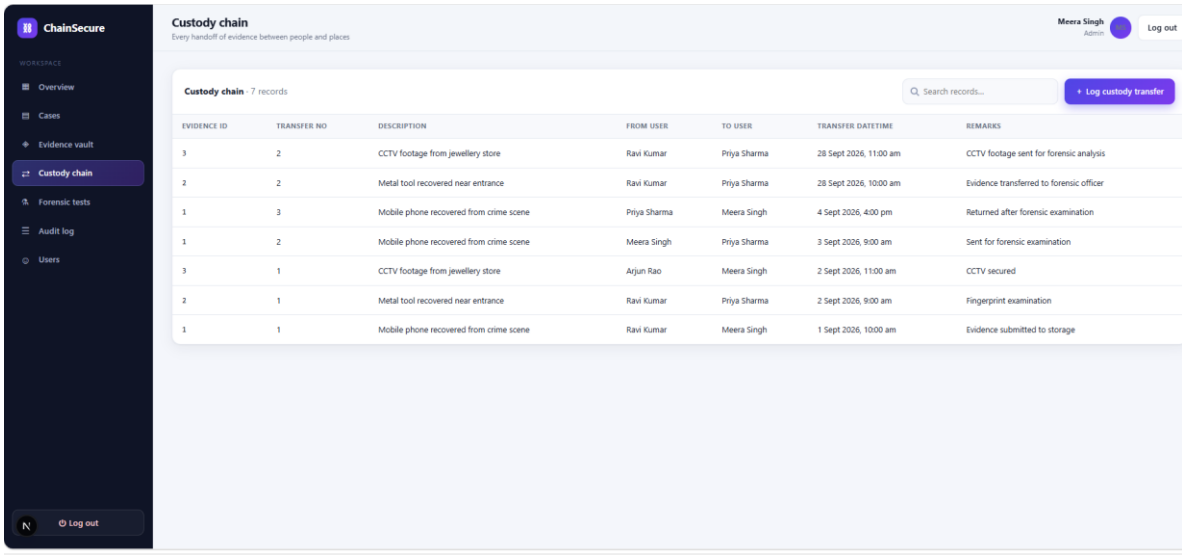


Figure 9.9: Custody chain page

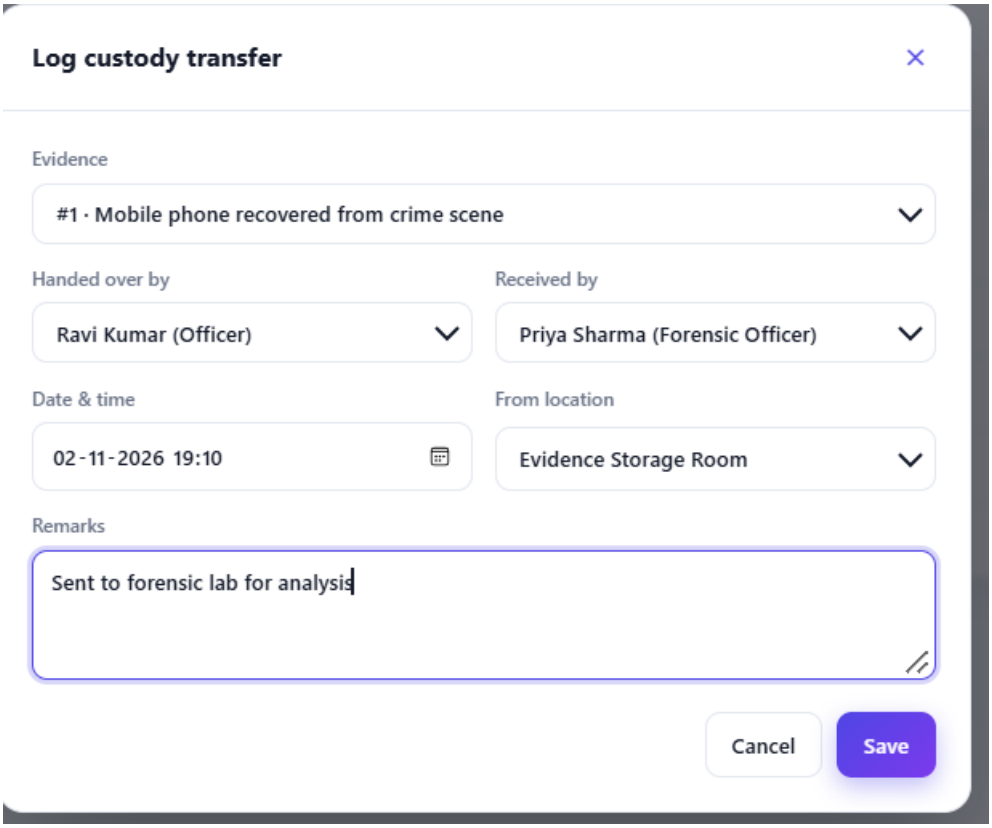


Figure 9.10: Log custody transfer form

9.4.6 Forensic tests

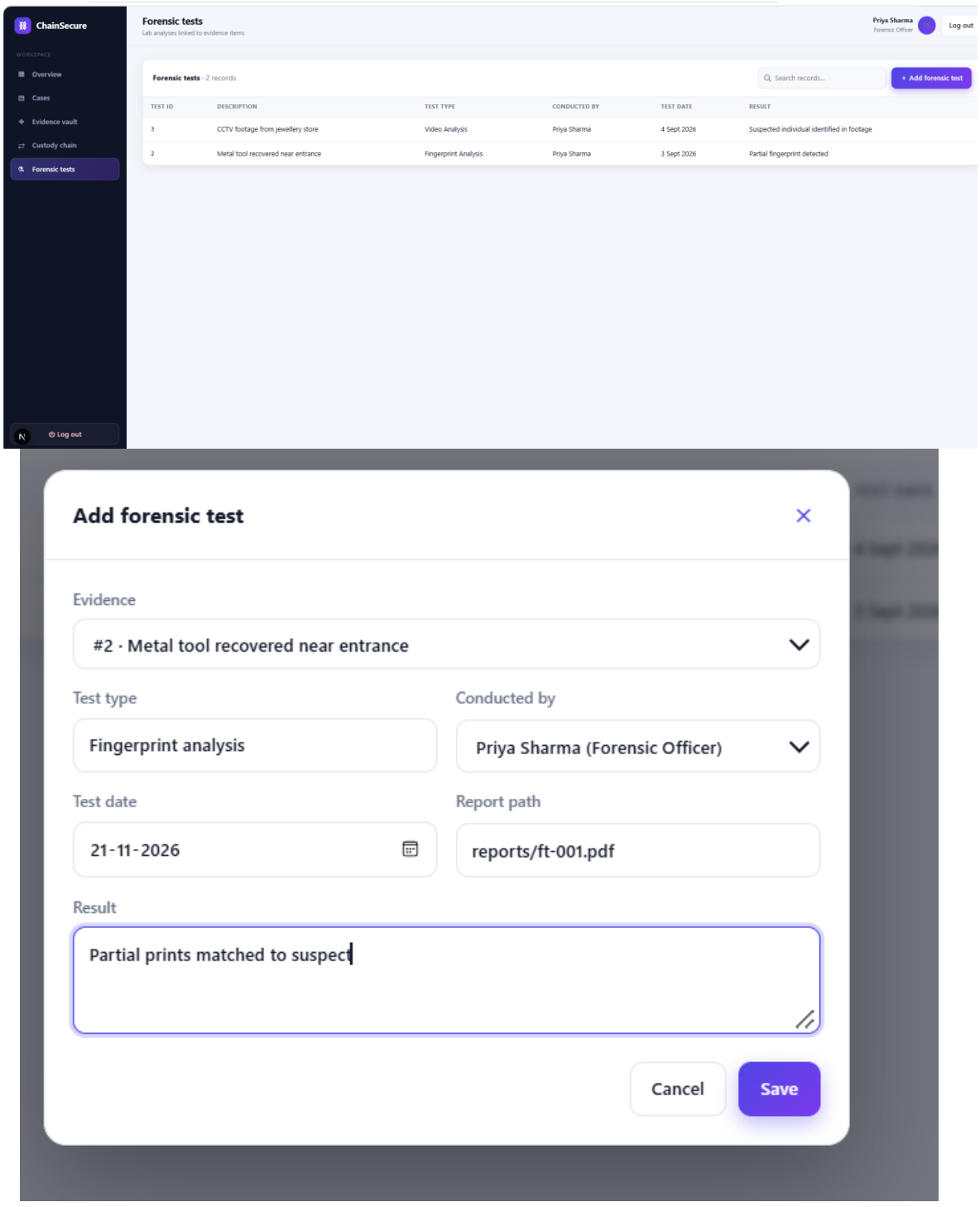


Figure 9.11: Forensic tests page and add-test form

9.4.7 Administration

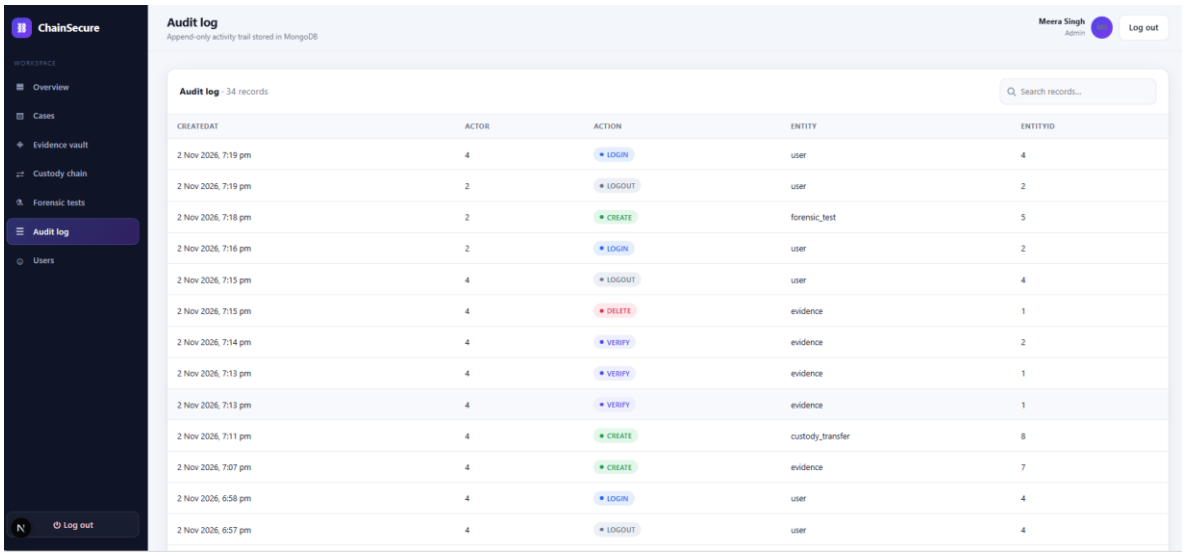


Figure 9.12: Audit log (Admin)

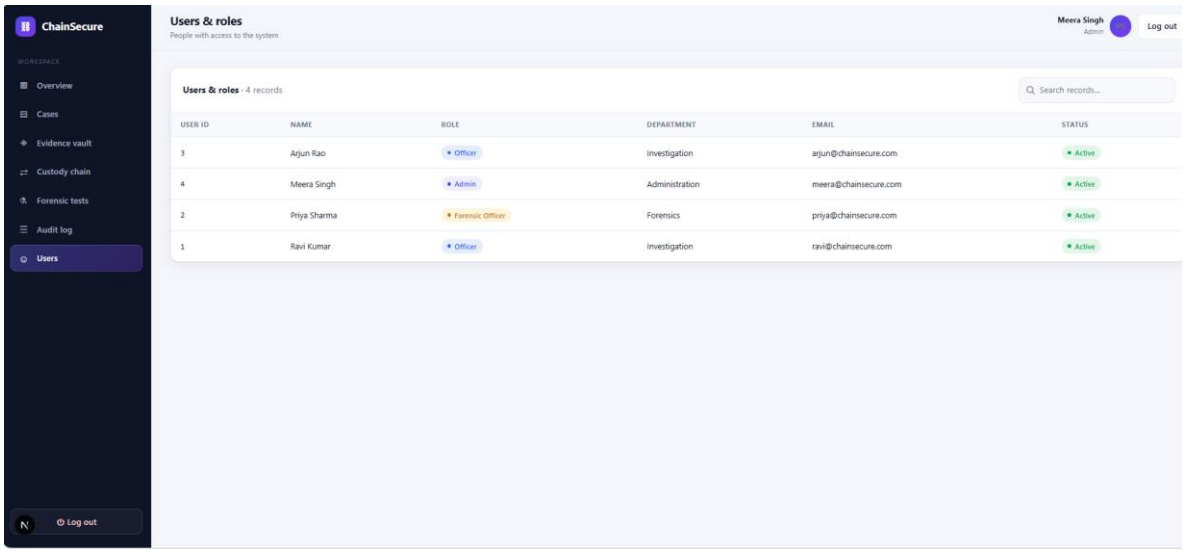


Figure 9.13: Users and roles (Admin)

9.4.8 Role-based access

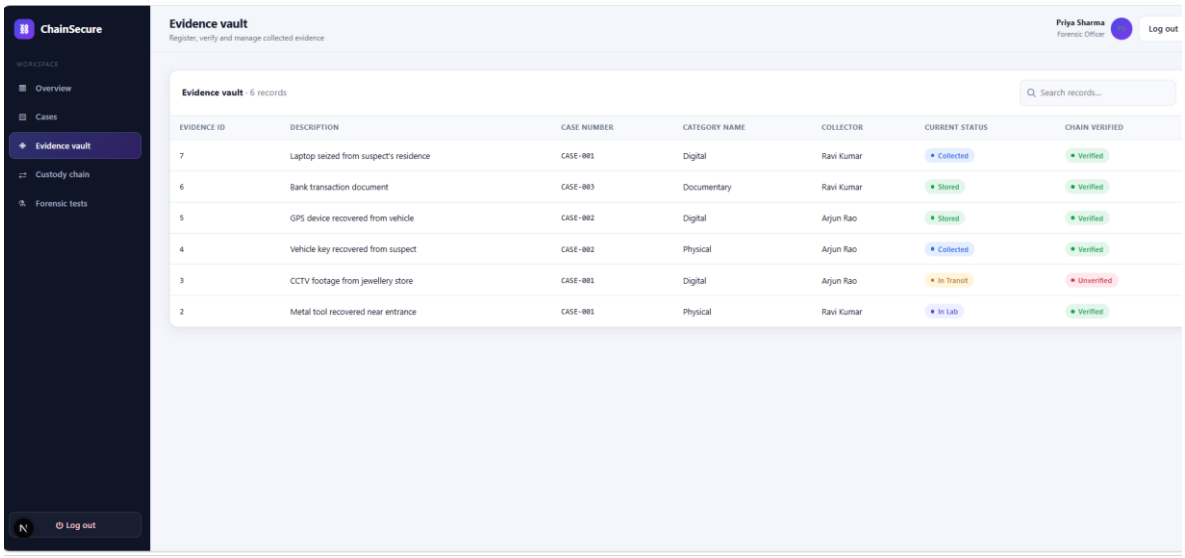


Figure 9.14: Restricted view for Forensic Officer

9.4.9 Logout

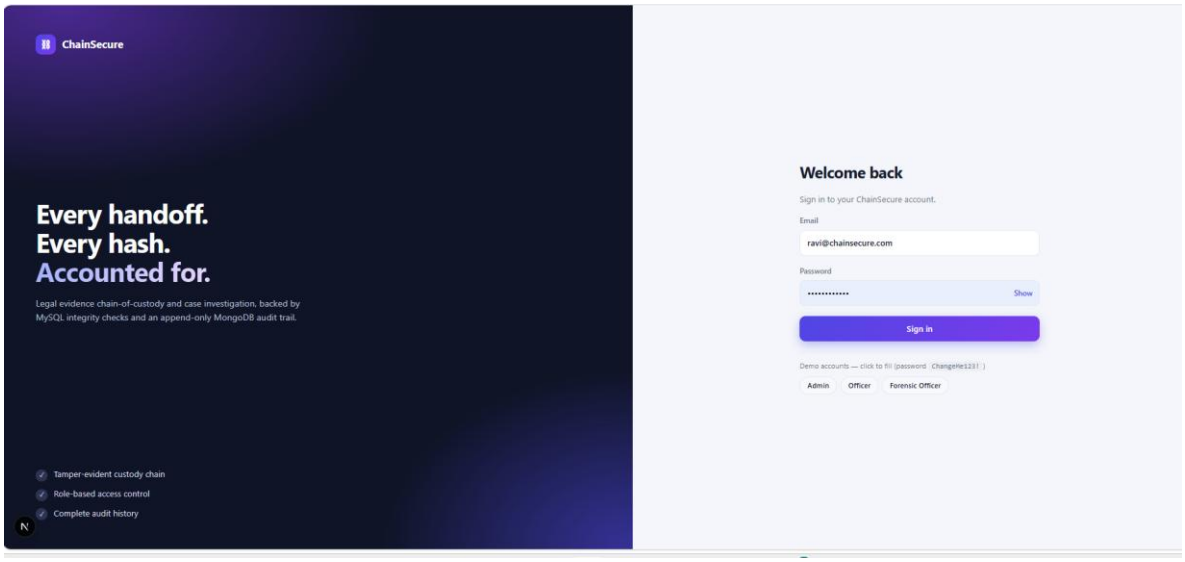


Figure 9.15: Logout

9.5 Audit trail in MongoDB

Each login, logout, creation, deletion and chain verification is stored as a document in the audit_events collection of the chainsecure_audit database, with the actor, action, entity, entity id and timestamp. The collection is only ever appended to by the application.

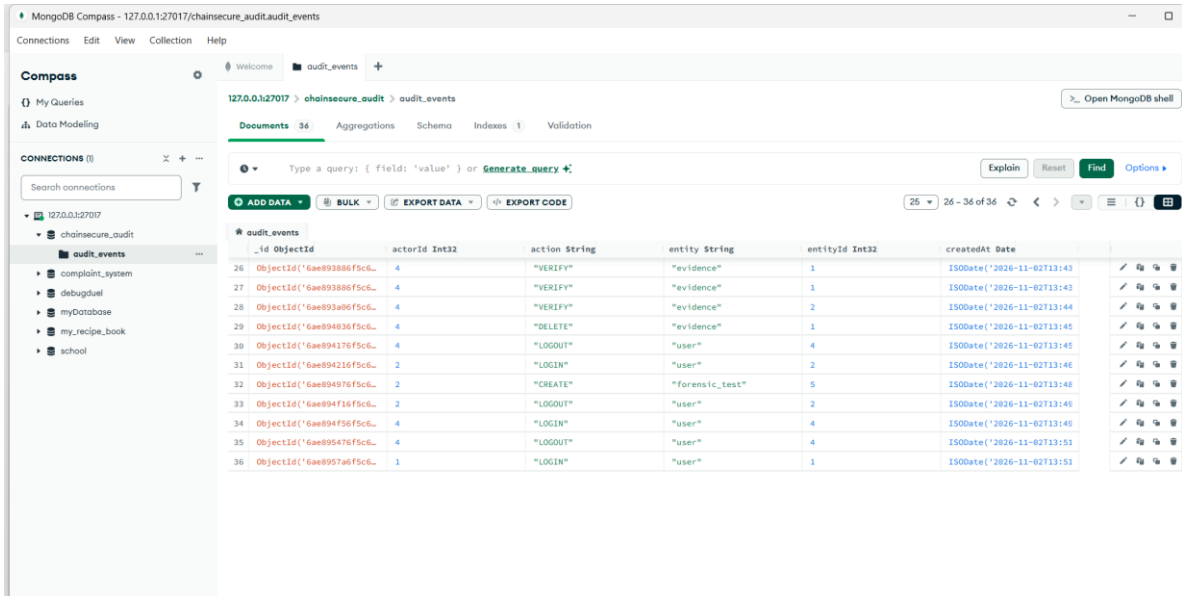


Figure 9.16: MongoDB audit_events collection

9.6 Testing

The following test cases were prepared for the application. Execute each one and tick the result column.

Table 9.2: Application test cases

No.	Test case	Expected result	Result
1	Log in with a wrong password	Error “Invalid credentials”; no session created	
2	Log in as Admin with the correct password	Redirected to the dashboard; LOGIN event stored in MongoDB	
3	Click Log out	Session cookie cleared; login page shown; LOGOUT event stored	
4	Open the dashboard URL after logging out	Redirected to the login page	
5	Register evidence with an invalid hash	Validation error (hash must be 64 hexadecimal characters)	
6	Register evidence with valid data	Saved; appears in the evidence vault; CREATE event stored	
7	Log a custody transfer	Transfer number allocated; trigger sets evidence to In_Transit and clears Chain_Verified	
8	Click Verify on that evidence	Function returns VALID; chain badge becomes Verified	
9	Forensic Officer tries to register evidence (API call)	403 – role cannot register evidence	
10	Non-admin opens the audit log	Audit and Users menus hidden; API returns 403	

9.7 Conclusion and future enhancements

ChainSecure shows how a relational database can enforce the rules of an evidence chain of custody directly in the data layer – through constraints, a trigger, a verification function, a view, indexes and transactions – while a second database provides an independent audit trail. The Next.js interface makes these features usable by Admins, Officers and Forensic Officers through secure, role-based screens.

Possible future enhancements include:

- Multi-factor authentication and institutional single sign-on.
- Object storage for forensic report files and evidence attachments.
- Complete case, person and user management screens.
- Cryptographic signing of each custody event.
- Analytics dashboards and similarity search over evidence descriptions.

REFERENCES

[1] MySQL 8.0 Reference Manual, Oracle Corporation. https://dev.mysql.com/doc/refman/8.0/en/

[2] MongoDB Manual, MongoDB Inc. https://www.mongodb.com/docs/manual/

[3] Next.js Documentation, Vercel. https://nextjs.org/docs

[4] React Documentation. https://react.dev

[5] Zod – TypeScript-first schema validation. https://zod.dev

[6] jose – JSON Web Token library. https://github.com/panva/jose

[7] bcrypt.js – password hashing library. https://github.com/dcodeIO/bcrypt.js

[8] R. Elmasri and S. Navathe, Fundamentals of Database Systems, Pearson.

GitHub Link: Vaniprsy/CS661_CS665_dbms-miniproject

APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS

Term	Meaning
DBMS	Database Management System
ER	Entity–Relationship (model / diagram)
DDL	Data Definition Language (CREATE, ALTER, DROP)
DML	Data Manipulation Language (INSERT, SELECT, UPDATE, DELETE)
PK / FK	Primary key / Foreign key
SQL	Structured Query Language
Chain of custody	Chronological record of everyone who handled an evidence item
Hash value	Fixed-length digest (e.g. SHA-256) used to detect any change to evidence
Trigger	SQL code that runs automatically when a table is modified
View	Stored query that behaves like a virtual table
JWT	JSON Web Token – signed token used for the login session
RBAC	Role-Based Access Control
API	Application Programming Interface
bcrypt	Adaptive password-hashing algorithm
SRN	Student Registration Number